

변경분 — signus 필사용 코드북 바뀐 파일은 통째로 실었습니다 · 초록 줄만 달라진 곳

git HEAD → 2026-07-27 c6a9bc6+수정중 | 11개 파일 · +1331 / -42 줄 · 총 45쪽 | 새 코드북 77쪽

변경된 파일 (괄호=쪽) pyproject.toml(1) · signus/sigio.py(1) · signus/triage.py(4) · signus/dsp.py(5)
 signus/fsk.py(10) · signus/chirp.py(14) · signus/pipeline.py(16) · signus/server.py(25) · signus/web/index.html(26)
 signus/web/style.css(29) · signus/web/app.js(33)

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · 붉은 점선 = 그 자리에서 몇 줄 사라짐 · L, = 윗줄에 붙는 이어짐

pyproject.toml +3 · 코드북 2쪽 · 새로 쓸 줄 24-26

```

1 1  [build-system]
2 2  requires = ["setuptools>=68"]
3 3  build-backend = "setuptools.build_meta"
4 4
5 5  [project]
6 6  name = "signus"
7 7  version = "2.0.0"
8 8  description = "Blind PSK/QAM demodulator: auto parameter detection + constellation UI"
9 9  requires-python = ">=3.11"
10 10 dependencies = ["numpy>=1.26", "scipy>=1.11"]
11 11
12 12 [project.optional-dependencies]
13 13 dev = ["pytest>=8.0", "ruff>=0.4"]
14 14 # Optional JIT for the sequential equalizer/carrier loops (16-34x on those loops).
15 15 # Falls back to identical pure-numpy when absent. NOTE: numba pins numpy<2.5.
16 16 fast = ["numba>=0.60"]
17 17
18 18 [project.scripts]
19 19 signus = "signus.cli:main"
20 20
21 21 [tool.setuptools.packages.find]
22 22 include = ["signus*"]
23 23
24 + [tool.setuptools.package-data]
25 + signus = ["web/*"] # the served frontend: without this a pip install 404s the whole UI
26 +
24 27 [tool.pytest.ini_options]
25 28 pythonpath = ["."]
26 29 testpaths = ["tests"]
27 30 addopts = "-q"
28 31 markers = ["stretch: slow/marginal cases, opt-in"]
29 32
30 33 [tool.ruff]
31 34 target-version = "py311"
32 35 line-length = 100
33 36
34 37 [tool.ruff.lint]
35 38 select = ["E", "F", "I", "UP", "B", "SIM", "NPY"]

```

signus/sigio.py +11 -4 · 코드북 4쪽 · 새로 쓸 줄 69-75, 78-81

```

1 1  """Sample file I/O. Filename carries read-necessities ONLY (fs, iq|real, sample
2 2  type, endian, bitrev); ground truth lives in a `<filename>.json` sidecar the
3 3  analyzer never reads. SigMF sidecars (`<stem>.sigmf-meta`) are also understood."""
4 4
5 5  import json
6 6  import os
7 7  import re
8 8  from dataclasses import dataclass
9 9
10 10 import numpy as np
11 11
12 12 _FS_TOK = re.compile(r"(\.?^|_)fs(\d+(?:(?:\.\d+)?(?:e[+-]?\d+)?)(?:_|$)", re.I)

```

```

13 13 _RF_TOK = re.compile(r"(?:^|_)rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?)?(?:_|$)", re.I)
14 14
15 15 # token -> (numpy code, full-scale divisor, offset). u-types are offset binary
16 16 # (e.g. 80 / RTL-SDR u8: 0..255 with 128 = zero).
17 17 _DTYPES = {
18 18     "i8": ("i1", 128.0, 0.0), "u8": ("u1", 128.0, 128.0),
19 19     "i16": ("i2", 32768.0, 0.0), "u16": ("u2", 32768.0, 32768.0),
20 20     "f32": ("f4", 1.0, 0.0), "f64": ("f8", 1.0, 0.0),
21 21 }
22 22 # baudline-style aliases: <bits>t = two's complement, <bits>o = offset binary
23 23 _ALIAS = {"8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64"}
24 24 _BITREV = np.array([int(f"{i:08b}"[::-1]), 2] for i in range(256)], dtype=np.uint8)
25 25
26 26 # SigMF core:datatype -> (dtype token, fmt, endian)
27 27 _SIGMF = {"cf32": ("f32", "iq"), "ci16": ("i16", "iq"), "ci8": ("i8", "iq"),
28 28     "cu8": ("u8", "iq"), "rf32": ("f32", "real"), "ri16": ("i16", "real")}
29 29
30 30
31 31 @dataclass
32 32 class Meta:
33 33     fs: float | None = None
34 34     fmt: str | None = None # 'iq' | 'real'
35 35     dtype: str = "i16" # key of _DTYPES
36 36     endian: str = "le" # 'le' | 'be' (floats included)
37 37     bitrev: bool = False # bit order reversed within each byte
38 38     rf_center: float | None = None # RF centre freq (Hz); None = unknown (report baseband)
39 39
40 40     def ok(self) -> bool:
41 41         return self.fs is not None and self.fmt in ("iq", "real") and self.dtype in _DTYPES
42 42
43 43
44 44 def parse_name(name: str) -> Meta:
45 45     """Extract fs/fmt/dtype/endian/bitrev tokens from a filename; rest ignored."""
46 46     stem = os.path.splitext(os.path.basename(name))[0].lower()
47 47     m = Meta()
48 48     if mt := _FS_TOK.search(stem):
49 49         m.fs = float(mt.group(1))
50 50     if rt := _RF_TOK.search(stem):
51 51         m.rf_center = float(rt.group(1))
52 52     toks = stem.split("_")
53 53     m.fmt = next((t for t in toks if t in ("iq", "real")), None)
54 54     m.dtype = next((_ALIAS.get(t, t) for t in toks if t in _DTYPES or t in _ALIAS), "i16")
55 55     m.endian = "be" if "be" in toks else "le"
56 56     m.bitrev = "bitrev" in toks
57 57     return m
58 58
59 59
60 60 def parse_sigmf(path: str) -> Meta | None:
61 61     """Read `<stem>.sigmf-meta` (core:sample_rate + core:datatype) if present."""
62 62     try:
63 63         with open(os.path.splitext(path)[0] + ".sigmf-meta") as fh:
64 64             doc = json.load(fh)
65 65             g = doc["global"]
66 66             code = g["core:datatype"].replace("_le", "").replace("_be", "")
67 67             dtype, fmt = _SIGMF[code]
68 68             endian = "be" if g["core:datatype"].endswith("_be") else "le"
69 69             fs = float(g["core:sample_rate"])
70 70     except (OSError, KeyError, ValueError, TypeError, AttributeError):
71 71         return None # MANDATORY fields malformed: fall back to filename tokens
72 72     try:

```

```

73 + # rf centre is OPTIONAL: a malformed value must cost only this field, never the whole
74 + # sidecar -- discarding valid fs/fmt/dtype here silently re-read the file with filename
75 + # tokens, which can lie about fs (a quiet garbage decode).
69 76 caps = doc.get("captures") or []
70 77 rf = caps[0].get("core:frequency") if caps else None
78 + rf = None if rf is None else float(rf)
79 + except (KeyError, ValueError, TypeError, AttributeError, IndexError):
80 + rf = None
81 + return Meta(fs, fmt, dtype, endian, rf_center=rf)
75 82
76 83
77 84 def make_name(label: str, m: Meta, ext: str) -> str:
78 85 fs_tok = str(int(m.fs)) if m.fs == int(m.fs) else f"{m.fs:f".rstrip("0").rstrip(".")
79 86 extra = ("_be" if m.endian == "be" else "") + ("_bitrev" if m.bitrev else "")
80 87 return f"{label}_fs{fs_tok}_{m.fmt}_{m.dtype}{extra}.{ext}"
81 88
82 89
83 90 def decode(raw: bytes, meta: Meta) -> np.ndarray:
84 91 """Raw bytes -> samples: complex128 for 'iq', float64 for 'real'.
85 92 Tolerates truncated files (drops trailing partial samples)."""
86 93 code, scale, off = _DTYPES[meta.dtype]
87 94 if meta.bitrev:
88 95 raw = _BITREV[np.frombuffer(raw, dtype=np.uint8)].tobytes()
89 96 step = np.dtype(code).itemsize
90 97 dt = ("<" if meta.endian == "le" else ">") + code
91 98 x = np.frombuffer(raw[: len(raw) - len(raw) % step], dtype=dt).astype(np.float64)
92 99 x = (x - off) / scale
93 100 if meta.fmt == "iq":
94 101 x = x[: x.size & ~1] # drop dangling half-sample
95 102 return x[0::2] + 1j * x[1::2]
96 103 return x
97 104
98 105
99 106 def read(path: str, meta: Meta | None = None) -> tuple[np.ndarray, Meta]:
100 107 meta = meta or parse_sigmf(path) or parse_name(path)
101 108 if not meta.ok():
102 109 raise ValueError(f"need fs + fmt (filename tokens, SigMF, or --fs/--fmt): {path}")
103 110 with open(path, "rb") as fh:
104 111 x = decode(fh.read(), meta)
105 112 if x.size < 256:
106 113 raise ValueError(f"신호가 너무 짧습니다 ({x.size} samples): {path}")
107 114 return x, meta
108 115
109 116
110 117 def write(path: str, x: np.ndarray, meta: Meta) -> None:
111 118 """Quantize (ints: scaled to 99.9th-percentile full scale) and write interleaved."""
112 119 code, scale, off = _DTYPES[meta.dtype]
113 120 flat = np.column_stack([x.real, x.imag]).ravel() if np.iscomplexobj(x) else x.real
114 121 if scale != 1.0: # integer types
115 122 peak = np.percentile(np.abs(flat), 99.9) or 1.0
116 123 flat = np.round(flat * (scale - 1) / peak) + off
117 124 info = np.iinfo(code)
118 125 flat = np.clip(flat, info.min, info.max)
119 126 out = flat.astype(("<" if meta.endian == "le" else ">") + code)
120 127 if meta.bitrev:
121 128 out = np.frombuffer(_BITREV[np.frombuffer(out.tobytes(), np.uint8)].tobytes(),
122 129 dtype=out.dtype)
123 130 out.tofile(path)
124 131
125 132

```

```

126 133 def sidecar_write(path: str, meta: Meta, truth: dict, gen: dict) -> None:
127 134     doc = {"fs": meta.fs, "fmt": meta.fmt, "dtype": meta.dtype,
128 135           "endian": meta.endian, "bitrev": meta.bitrev, "truth": truth, "gen": gen}
129 136     with open(path + ".json", "w") as fh:
130 137         json.dump(doc, fh, indent=1)
131 138
132 139
133 140 def sidecar_read(path: str) -> dict | None:
134 141     try:
135 142         with open(path + ".json") as fh:
136 143             return json.load(fh)
137 144     except (OSError, ValueError):
138 145         return None

```

signus/triage.py +24 -2 · 코드북 17쪽 · 새로 쓸 줄 28-42, 46, 55-62

```

1 1  """Per-channel triage: decide whether an extracted channel is a demodulable digital
2 2  signal or something the demodulator must NOT force onto a constellation (analog FM
3 3  voice, a CW tone/spur). Envelope CV is the SNR-robust separator: RRC-shaped PSK/QAM
4 4  always sit at CV>=0.25, while FSK/FM/CW are constant-envelope (CV<=0.05)."""
5 5
6 6  import numpy as np
7 7
8 8  from .chirp import is_chirp, sweeps_band
9 9  from .fsk import fsk_gate
10 10
11 11  _CV_CE = 0.15      # constant-envelope ceiling: below it a non-FSK signal is analog/tone
12 12  _TONE_PAR = 500.0  # M-th-power peak-to-mean above which a constant-envelope signal is CW
13 13
14 14
15 15  def _carrier_par(x: np.ndarray, powers: tuple[int, ...] = (1, 2, 4, 8)) -> float:
16 16      """Best peak-to-mean of the M-th-power magnitude spectrum. A CW tone spikes at
17 17      p=1; analog FM and noise stay flat at every power."""
18 18      x = x - x.mean()
19 19      nfft = 1 << int(np.ceil(np.log2(x.size)))
20 20      win = np.hanning(x.size)
21 21      best = 0.0
22 22      for p in powers:
23 23         spec = np.abs(np.fft.fft(x ** p * win, nfft))
24 24         best = max(best, float(spec.max() / (spec.mean() + 1e-30)))
25 25      return best
26 26
27 27
28 + def _active(x: np.ndarray) -> np.ndarray:
29 +     """Trim leading/trailing dead air (envelope below half the top-quartile median). A bursty
30 +     channel's dead air corrupts BOTH triage discriminators -- it inflates the envelope CV past
31 +     fsk_gate's ceiling AND pollutes sweeps_band's IF histogram -- so every gate below must see
32 +     the emitter, not the silence around it. A continuous channel is returned unchanged."""
33 +     env = np.abs(x)
34 +     top = env[env > np.percentile(env, 75)]
35 +     if top.size == 0:
36 +         return x
37 +     on = np.flatnonzero(env > 0.5 * np.median(top))
38 +     if on.size < 256:
39 +         return x
40 +     return x[on[0]:on[-1] + 1]
41 +
42 +
28 43 def family(x: np.ndarray, fs: float) -> str:
29 44     """One of 'fsk' | 'chirp' | 'linear' | 'analog' | 'tone'. 'linear'/'fsk' go to the
30 45     demod; 'chirp'/'analog'/'tone' are reported as-is (never force-fit to a constellation)."""

```

```

46 + x = _active(x)
31 47 if fsk_gate(x, fs):
32 48     # a linear FMCW chirp / CSS (LoRa) trips fsk_gate too -- its swept IF reads bimodal to
33 49     # the gate -- and would then be force-demodulated into confident garbage FSK symbols.
34 50     # is_chirp fires on both a chirp and (at some h/ baud) real FSK, so the IF-SWEEP test
35 51     # breaks the tie: a genuine band-sweep is characterized as chirp, real FSK stays FSK.
36 52     if is_chirp(x, fs) and sweeps_band(x, fs):
37 53         return "chirp"
38 54     return "fsk"

55 + if is_chirp(x, fs) and sweeps_band(x, fs):
56 +     # linear chirp / CSS (LoRa) -- constant-envelope, would otherwise fall through to
57 +     # 'analog' (no M-power tone). The tie-break is REQUIRED here too: a bursty integer-h
58 +     # FSK channel (dead air inflates envelope CV past fsk_gate) also trips is_chirp via
59 +     # its CPFSK beat tones -- without sweeps_band it was labeled 'chirp' and never
60 +     # demodulated. A non-sweeping channel falls through, and the demod's burst isolation
61 +     # recovers the FSK.
62 +     return "chirp"

41 63 a = np.abs(x)
42 64 if a.std() / (a.mean() + 1e-12) < _CV_CE: # constant envelope, not FSK
43 65     return "tone" if _carrier_par(x) >= _TONE_PAR else "analog"
44 66     return "linear"

```

signus/dsp.py +27 -4 · 코드북 19쪽 · 새로 쓸 줄 60-65, 71-72, 101-105, 112-125

```

1 1 """Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
2 2 anti-shared-bug rule this imports only constellation reference data, never gen.py."""
3 3
4 4 from fractions import Fraction
5 5
6 6 import numpy as np
7 7 from scipy.ndimage import uniform_filter1d
8 8 from scipy.signal import hilbert, resample_poly, welch
9 9
10 10 from ._accel import dd_carrier
11 11 from .constellations import ideal_points
12 12
13 13
14 14 def analytic(x: np.ndarray) -> np.ndarray:
15 15     """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
16 16     samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
17 17     overflows on the |x|^2 the estimators take)."""
18 18     x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
19 19     x = np.where(np.abs(x) > 1e150, 0.0, x) # a finite spike that overflows |x|^2 is corrupt t
20 20     oo
21 21     if np.iscomplexobj(x):
22 22         return x.astype(np.complex128)
23 23     return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)
24 24
25 25 def _parab(ydb: np.ndarray, k: int) -> float:
26 26     """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
27 27     if k <= 0 or k >= ydb.size - 1:
28 28         return 0.0
29 29     a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
30 30     d = a - 2 * b + c
31 31     # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
32 32     # (d~0) or non-max triple can explode the ratio and drive baud negative -> a resample crash
33 33     return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0
34 34
35 35

```

```

36 36 def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
37 37     """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
38 38     Assignment is a running min over the k centers (no (N, k) broadcast temporary);
39 39     strict '<' keeps the lowest-index center on ties, so labels are identical to a
40 40     broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
41 41     c = np.quantile(a, (np.arange(k) + 0.5) / k)
42 42     lab = np.zeros(a.size, dtype=int)
43 43     for _ in range(iters):
44 44         d = np.abs(a - c[0])
45 45         lab = np.zeros(a.size, dtype=int)
46 46         for j in range(1, k):
47 47             dj = np.abs(a - c[j])
48 48             closer = dj < d
49 49             lab[closer] = j
50 50             d[closer] = dj[closer]
51 51         nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
52 52         if np.allclose(nc, c):
53 53             break
54 54         c = nc
55 55     return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))
56 56
57 57
58 58 # --- burst detection -----
59 59
60 + _SPIKE_PAR = 60.0 # raw peak-to-mean power above which a candidate run is an impulse smear, no
61 + t a
62 + # burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/Q
63 + AM
64 + # peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared o
65 + ver
66 + # a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with marg
67 + in.
68 +
69 +
70 70
71 + def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
72 + """Dual-threshold energy detector with an Otsu log-power floor; return every
73 merged burst in time order, or [(0, size)] when nothing stands out."""
74 n = x.size
75 win = max(64, n // 1000)
76 pw = np.abs(x) ** 2
77 ps = uniform_filter1d(pw, win)
78 lp = np.log10(ps + 1e-20)
79
80 # Otsu split of the log-power histogram: a floor that survives an 80%-full
81 # record because it is drawn from the (small) below-threshold noise class.
82 hist, edges = np.histogram(lp, bins=128)
83 centers = (edges[:-1] + edges[1:]) / 2
84 w = np.cumsum(hist).astype(float)
85 m = np.cumsum(hist * centers)
86 mb = m / np.where(w > 0, w, 1)
87 mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
88 btw = w * (w[-1] - w) * (mb - mf) ** 2
89 thr = centers[int(np.argmax(btw))]
90 noise = lp[lp < thr]
91 floor = noise.mean() if noise.size else lp.min()
92
93 above_hi = lp >= floor + 1.0 # 10 dB (power decade) over floor
94 above_lo = lp >= floor + 0.6 # 6 dB
95 dif = np.diff(above_lo.astype(np.int8))
96 starts = list(np.where(dif == 1)[0] + 1)

```

```

85 92     ends = list(np.where(dif == -1)[0] + 1)
86 93     if above_lo[0]:
87 94         starts.insert(0, 0)
88 95     if above_lo[-1]:
89 96         ends.append(n)
90 97     runs = [(s, e) for s, e in zip(starts, ends, strict=True) if above_hi[s:e].any()]
91 98     if not runs:
92 99         return [(0, n)]
93 100
101 +     gap = max(256, n // 200)          # merge gap heals intra-signal splits: proportional is f
    +     ine
102 +     mlen = max(256, min(n // 200, 1024)) # ...but the MIN burst length must not scale with the
103 +     # record: a short packetized burst (preamble-sync target, ~1k samples) in a long capture was
104 +     # dropped by n//200 and the [(0,n)] fallback buried it in noise. 1024 keeps the blip filter
105 +     # for records up to ~200k while capping it where the packet population starts.
95 106     merged = [list(runs[0])]
96 107     for s, e in runs[1:]:
97 108         if s - merged[-1][1] < gap:
98 109             merged[-1][1] = e
99 110         else:
100 111             merged.append([s, e])
112 +     # The mlen cap alone lets a short high-energy transient survive in a long record: the smooth
    +     er
113 +     # spreads a single impulse into a ~win-wide run that clears 1024, and its energy can outrank
114 +     # the real signal so analyze() auto-selects the blip. A modulated burst has a near-flat
115 +     # envelope (raw peak/mean power ~ a few); an impulse smear is one spike over noise (ratio ~w
    +     in).
116 +     # Reject runs whose RAW power is spike-dominated -- this separates them by shape, at any len
    +     gth.
117 +     out = []
118 +     for s, e in merged:
119 +         if e - s < mlen:
120 +             continue
121 +         seg = pw[s:e]
122 +         if float(seg.max()) / (seg.mean() + 1e-30) > _SPIKE_PAR:
123 +             continue
124 +         out.append((int(s), int(e)))
125 +     return out or [(0, n)]
103 126
104 127
105 128 def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
106 129     """The most energetic burst of find_bursts."""
107 130     return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))
108 131
109 132
110 133 # --- carrier estimation -----
111 134
112 135 def est_carrier(
113 136     x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
114 137     nfft: int = 65536, blocks: int = 4,
115 138 ) -> tuple[float, int, bool]:
116 139     """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
117 140     nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
118 141     freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))
119 142     best_par, best_s, sym = -1.0, None, powers[0]
120 143     for p in powers:
121 144         xp = x ** p
122 145         blk = xp.size // blocks
123 146         segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
124 147         acc = np.zeros(nfft)

```

```

125 148         for s in segs:
126 149             acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
127 150         spec = acc / len(segs)
128 151         par = spec.max() / spec.mean()
129 152         if par > best_par:
130 153             best_par, best_s, sym = par, spec, p
131 154
132 155     if best_s is None: # degenerate input (all-zero / no energy): no M-th-power tone exists
133 156         return 0.0, int(powers[0]), True
134 157     ydb = 20 * np.log10(best_s + 1e-20)
135 158     k = int(np.argmax(best_s))
136 159     f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
137 160     fc = f_peak / sym
138 161     ambiguous = abs(sym * fc) > 0.4 * fs
139 162     return float(fc), int(sym), bool(ambiguous)
140 163
141 164
142 165 def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
143 166     """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
144 167     whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power
145 168     (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
146 169     straddling +-fs/2 corrupts the two-sided centroid (bpsk fc=0.495*fs picked fc=-5000)."""
147 170     f, pxx = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
148 171     p = np.maximum(pxx - np.median(pxx), 0)
149 172     # range must reach +-fs/2 for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
150 173     # 8psk beyond 0.3125*fs with no candidate).
151 174     cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)]
152 175     if abs(fc + k * fs / sym) < 0.5 * fs]
153 176     half = fs / (2 * sym) # each candidate owns a fs/sym-wide cell
154 177     return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum
155 178         ()))
156 179
157 180 def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:
158 181     """Downconvert by fc (complex heterodyne)."""
159 182     return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)
160 183
161 184
162 185 # --- symbol rate + rolloff -----
163 186
164 187 def est_baud(
165 188     x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
166 189     blocks: int = 4,
167 190 ) -> tuple[float, float]:
168 191     """Cyclostationary line at the symbol rate in |x|^2; returns (baud, confidence).
169 192     Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
170 193     a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
171 194     u = np.abs(x) ** 2
172 195     u = u - u.mean()
173 196     blk = u.size // blocks
174 197     segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
175 198     nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs))))))
176 199     acc = np.zeros(nfft // 2 + 1)
177 200     for s in segs:
178 201         acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
179 202     spec = acc / len(segs)
180 203     freqs = np.fft.rfftfreq(nfft, 1 / fs)
181 204
182 205     lo = lo if lo is not None else 0.005 * fs
183 206     hi = hi if hi is not None else 0.45 * fs

```

```

184 207     band = (freqs >= lo) & (freqs <= hi)
185 208     if not band.any(): # degenerate window (e.g. noise-derived prior): default band
186 209         band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
187 210     idx = np.where(band)[0]
188 211     k = idx[int(np.argmax(spec[idx]))]
189 212     ydb = 20 * np.log10(spec + 1e-20)
190 213     baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
191 214     conf = spec[k] / (np.median(spec[band]) + 1e-20)
192 215     return float(baud), float(conf)
193 216
194 217
195 218 def occupied_bw(x: np.ndarray, fs: float) -> float:
196 219     """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way
197 220 from the 25th-percentile floor to the 90th-percentile in-band level, walk
198 221 outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
199 222 Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
200 223     f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
201 224     floor = np.percentile(pxx, 25)
202 225     order = np.argsort(np.abs(f))
203 226     idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
204 227     if idx.size == 0:
205 228         return 0.0
206 229     last = idx[-1]
207 230     return float(2 * np.abs(f[order][last]))
208 231
209 232
210 233 def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
211 234     """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
212 235     n = x.size
213 236     nper = min(8192, max(256, n // 8))
214 237     f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
215 238     f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
216 239     af = np.abs(f)
217 240     inband = np.median(pxx[af < 0.25 * baud])
218 241     noise = np.median(pxx[af > 0.9 * baud])
219 242     thr = noise + 0.10 * (inband - noise)
220 243     band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
221 244     if not band.any():
222 245         return 0.35
223 246     raw = 2 * af[band].max() / baud - 1
224 247     return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))
225 248
226 249
227 250 # --- resampling + matched filter -----
228 251
229 252 def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
230 253     """Rational resample to exactly sps samples/symbol."""
231 254     r = Fraction(sps * baud / fs).limit_denominator(2000)
232 255     return resample_poly(x, r.numerator, r.denominator)
233 256
234 257
235 258 def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
236 259     """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
237 260 |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""
238 261     m = (span * sps) // 2
239 262     t = np.arange(-m, m + 1) / sps
240 263     a = alpha
241 264     num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
242 265     den = np.pi * t * (1 - (4 * a * t) ** 2)
243 266     h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)

```

```

244 267         h[t == 0] = 1 - a + 4 * a / np.pi
245 268     if a > 0:
246 269         s = np.isclose(np.abs(t), 1 / (4 * a))
247 270         h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
248 271         + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
249 272     return h / np.sqrt(np.sum(h ** 2))
250 273
251 274
252 275 def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
253 276     """Apply the RX matched RRC filter."""
254 277     return np.convolve(x, _rx_rrc(alpha, span, sps), "same")
255 278
256 279
257 280 # --- timing recovery -----
258 281
259 282 def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
260 283     """Block-wise 0erder-Meyr timing recovery; returns `out` samples/symbol
261 284     (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
262 285     Per block the spectral-line phase gives the offset; unwrapped across blocks
263 286     to track clock drift, then interpolated per symbol."""
264 287     n = sps
265 288     nsym = x.size // n
266 289     if nsym < 1:
267 290         return x[:0].astype(np.complex128)
268 291     xt = x[:nsym * n]
269 292     p = np.abs(xt) ** 2
270 293     ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
271 294     bs = block * n
272 295     if xt.size <= bs: # single block: one global fractional offset
273 296         eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
274 297     else:
275 298         nblk = xt.size // bs
276 299         c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
277 300         eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)
278 301         eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
279 302         pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
280 303         idx = np.arange(xt.size)
281 304         return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)
282 305
283 306
284 307 # --- decision-directed carrier sync -----
285 308
286 309 def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarra
287 310     y:
288 311     """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
289 312     Costas loop, which jitters QAM at the correct points). Sequential feedback (each
290 313     phase update depends on the prior decision) -> numba kernel when available."""
291 314     pts = ideal_points(mod)
292 315     z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
293 316     return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
294 317                      np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)

```

```

signus/fsk.py +77 -13 · 코드북 33쪽 · 새로 쓸 줄 7, 10, 19-40, 60-64, 73, 75-80, 85-93, 119-141, 145-147, 149-154
1 1     """FSK/CPM receive path: constant-envelope gate + frequency-discriminator demod.
2 2     Runs on the DC-blocked analytic burst, before any carrier work (linear front-end
3 3     does not apply). Imports only constellation reference data (anti-shared-bug rule)."""
4 4
5 5     import numpy as np
6 6     from scipy.ndimage import uniform_filter1d
7 + from scipy.signal import firwin, resample_poly, welch

```

```

7 8
8 9 from .constellations import fsk_levels, to_bits
10 + from .dsp import _kmeans1d, _parab, analytic, mix
11 12
11 12 _CV_MAX = 0.24 # envelope coeff-of-variation ceiling. Recalibrated on a broad grid: real
12 13 # FSK (all h x SNR>=10 x high baud) tops out at cv 0.224, but constant-modulus PSK at very
13 14 # high baud (fs/baud~3) or near +-fs/2 dips to cv>=0.250 and used to slip the old 0.30 gate
14 15 # (false FSK). 0.24 sits in the 0.026 gap -> keeps every real FSK, rejects the PSK misfires.
15 16 _SEP_MIN = 3.1 # instantaneous-freq 2-means separation floor (FSK>=3.39, rest<=2.85)
16 17 _K4_RATIO = 2.6 # sp(k=2)/sp(k=4) collapse above which 4 levels beat 2
17 18 _MIN_SYMS = 8 # fewer symbols than this cannot cluster 2/4 levels (empty-quantile crash)
19 + _PROM_MIN = 10.0 # a symbol-rate line peak/median below this is unreliable -- BUT only confiden
  + t-
20 + # wrong when paired with too few symbols (a long low-index burst has a weak li
  + ne
21 + # yet a correct baud). The sweep sits at prom 23-73.
22 + _NSYM_MIN = 200 # ...so reject a weak line only below this symbol count. A 100 floor was tried
23 + # (to recover more short bursts) but an independent grid found weak-prominence
24 + # half-rate folds in the 100-199 band that still cluster tight enough to loc
  + k >=60
25 + # (msk h=0.5 baud 24-32% off, lock 61-65) -- confident-wrong. 200 keeps them o
  + ut;
26 + # precision over recall (the short-burst confident-wrong is the cardinal sin).
27 + _DECIM_FRAC = 0.30 # decimate a heavily-oversampled burst so the tones fill ~this of Nyquist
28 + _DECIM_MIN = 3 # ...but only when that needs a factor >= this, so the sps~10 demod grid
29 + # (fsk2/fsk4/msk baud=fs/10) is untouched -> the sweep stays byte-identical
30 +
31 +
32 + def _bw99(x: np.ndarray, fs: float) -> float:
33 + """Two-sided 99%-power occupied bandwidth (Hz), noise-pedestal subtracted."""
34 + f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
35 + f, p = np.fft.fftshift(f), np.fft.fftshift(p)
36 + p = np.maximum(p - np.median(p), 0.0)
37 + cs = np.cumsum(p) / (p.sum() + 1e-30)
38 + lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)] # clamp BOTH edges: a degenerate PSD
39 + hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)] # (all-zero after floor subtraction)
40 + return float(abs(hi - lo)) # otherwise indexes one past the end
18 41
19 42
20 43 def _dcblock(x: np.ndarray) -> np.ndarray:
21 44 """Complex128 analytic burst, DC-blocked (front-end convention)."""
22 45 x = analytic(x)
23 46 return x - x.mean()
24 47
25 48
26 49 def _ifreq(x: np.ndarray, fs: float) -> np.ndarray:
27 50 """Instantaneous frequency (Hz) from sample-to-sample phase differences."""
28 51 return np.angle(x[1:] * np.conj(x[:-1])) * fs / (2 * np.pi)
29 52
30 53
31 54 def _sep(a: np.ndarray) -> float:
32 55 """2-means between/within separation ratio of a 1-D sample set."""
33 56 c, _, sp = _kmeans1d(a, 2)
34 57 return abs(c[1] - c[0]) / (sp + 1e-9)
35 58
36 59
60 + def _est_baud(f: np.ndarray, fs: float) -> tuple[float, float]:
61 + """Symbol rate from the spectral line of |diff(smoothed f)| (transition impulses),
62 + harmonic-folded to the fundamental (2-level lines alias to 2*baud). Returns
63 + (baud, prominence) where prominence = peak/median of the in-band line: a weak line

```

```

64 + (few symbols, no clean symbol clock) means the baud is UNRELIABLE -> caller rejects."""
40 65 d = np.abs(np.diff(uniform_filter1d(f, 2)))
41 66 d = d - d.mean()
42 67 n = 1 << int(np.ceil(np.log2(d.size)))
43 68 spec = np.abs(np.fft.rfft(d * np.hanning(d.size), n))
44 69 freqs = np.fft.rfftfreq(n, 1 / fs)
45 70 df = freqs[1] - freqs[0]
46 71 lo, hi = 0.002 * fs, 0.45 * fs
47 72 idx = np.where((freqs >= lo) & (freqs <= hi))[0]
73 + prom = float(spec[idx].max() / (np.median(spec[idx]) + 1e-20)) if idx.size else 0.0
48 74 k = idx[int(np.argmax(spec[idx]))]

75 +
76 + def _hp(kk: int) -> float:                                # harmonic-comb energy: the TRUE fundamenta
77 +     l
78 +     return float(sum(spec[m * kk] for m in range(1, 6) if m * kk < spec.size)) # captures m
79 +     ost
80 +     # in-range harmonics ONLY: clamping out-of-range ones to the last bin multi-counted the
81 +     # Nyquist bin, inflating _hp at a 2*baud peak enough to veto the legitimate fold -> 2x bau
82 +     d.

49 81 for div in (3, 2):
50 82     ks = int(round(freqs[k] / div / df))
51 83     if freqs[ks] >= lo:
52 84         w = spec[max(0, ks - 2):ks + 3]

85 +         kk = max(0, ks - 2) + int(np.argmax(w)) if w.size else k
86 +         # fold to the sub-harmonic ONLY if it is genuinely the fundamental -- its harmonic c
87 +         omb
88 +         # must beat the current peak's. A real baud/2 fundamental captures MORE harmonic lin
89 +         es
90 +         # than the 2*baud peak; spurious low-h/low-SNR energy at baud/2 does not (it once
91 +         # slipped the 0.34 gate and folded baud -> baud/2: a confident WRONG decode). The co
92 +         mb
93 +         # guard vetoes that. On the sps~10 grid the fold never fired (0.34 unmet) -> sweep s
94 +         ame.
95 +         if w.size and w.max() > 0.34 * spec[k] and _hp(kk) >= _hp(k):
96 +             k = kk
97 +         return float(freqs[k] + _parab(20 * np.log10(spec + 1e-20), k) * df), prom

56 94
57 95
58 96 def fsk_gate(x: np.ndarray, fs: float) -> bool:
59 97     """True when the burst looks like FSK/CPM: near-constant envelope AND a bimodal
60 98     instantaneous frequency. Calibrated on generate() over all GEN_MODS x seeds 0..3
61 99     x snr {25,15}: every linear mod fails one test with margin -- bpsk/dbpsk have
62 100     cv~0.40 (>_CV_MAX), the rest have freq-sep<=2.85 (<_SEP_MIN); fsk2/fsk4/msk keep
63 101     cv<=0.22 and freq-sep>=3.39 down to snr 10 (margins ~0.08 and ~0.29).
64 102     Recenter by the mean instantaneous frequency first: at a large carrier offset a
65 103     linear mod's phase-transition spikes wrap past +-fs/2 and fake a second mode."""
66 104     x = _dcblock(x)
67 105     a = np.abs(x)
68 106     if a.std() / (a.mean() + 1e-12) >= _CV_MAX:
69 107         return False
70 108     f = _ifreq(x, fs)
71 109     x = x * np.exp(-2j * np.pi * np.mean(f) / fs * np.arange(x.size))
72 110     return bool(_sep(uniform_filter1d(_ifreq(x, fs), 4)) > _SEP_MIN)
73 111
74 112
75 113 def analyze_fsk(x: np.ndarray, fs: float) -> dict:
76 114     """Full FSK/MSK demod by frequency discrimination. Returns mod, fc, baud, h,
77 115     lock (0-100), level_freqs (Hz, sorted, centered), symbols (normalized f per
78 116     symbol) and Gray-demapped bits."""

```

```

79 117     x = _dcblock(x)
80 118     f = _ifreq(x, fs)

119 +     # A carrier OFFSET (tones sit at fc +/- dev, not around DC) corrupts the symbol-rate estimat
    +     e two
120 +     # ways -- off-centre it trips _est_baud's harmonic fold, and the tones fall outside the deci
    +     m
121 +     # LPF passband -- either way a confident WRONG baud. Recentre the discriminator on the carri
    +     er
122 +     # (mean instantaneous freq) first; only for a significant offset, so a centred burst
123 +     # (fc~0, the whole demod/sweep grid) is byte-identical. Levels are already centred downstrea
    +     m.
124 +     fc0 = float(np.mean(f))
125 +     if abs(fc0) > 0.005 * fs:
126 +         x = mix(x, fs, fc0)
127 +         f = _ifreq(x, fs)
128 +     else:
129 +         fc0 = 0.0
130 +     baud, prom = _est_baud(f, fs)
131 +     # heavy oversampling (fs/baud >> 10) buries the symbol-rate line under broadband transition
132 +     # artifacts -> _est_baud locks a spurious peak (baud ~25% off) while the clean tones still
133 +     # cluster (lock ~94): a confident WRONG decode. Re-estimate baud on a decimated copy (tone
    +     s ->
134 +     # ~_DECIM_FRAC of Nyquist, as the survey channelizer does). Gated at _DECIM_MIN so the sps~1
    +     0
135 +     # grid is untouched (fsk2/fsk4/msk baud=fs/10 -> d<3 -> no re-estimate -> sweep byte-identic
    +     al).
136 +     bw = _bw99(x, fs)
137 +     d = int(_DECIM_FRAC * fs / bw) if bw > 0 else 1
138 +     if d >= _DECIM_MIN:
139 +         xd = np.convolve(x, firwin(129, min(0.9 * (fs / d) / 2, bw) / (fs / 2)), "same")
140 +         baud, prom = _est_baud(_ifreq(resample_poly(xd, 1, d), fs / d), fs / d)
141 +     if baud <= 0:
83 142         raise ValueError("FSK 버스트에서 심볼율을 추정할 수 없습니다 (너무 짧음)")
84 143     sps = fs / baud
85 144     f = uniform_filter1d(f, max(1, int(round(sps / 2))))

145 +     resid = float(f.mean())
146 +     fc = fc0 + resid                                # absolute carrier = recentre offset + in-band residual
147 +     fcen = f - resid

88 148     nsym = int(fcen.size / sps) - 1

149 +     # a WEAK symbol-rate line (prom < _PROM_MIN) is unreliable ONLY when symbols are also too fe
    +     w to
150 +     # average (nsym < _NSYM_MIN): a short burst's spurious peak still clusters tight (few point
    +     s ->
151 +     # high lock) = a confident WRONG baud. A long low-index (h~0.35) burst also has a weak lin
    +     e but
152 +     # MANY symbols -> its baud is correct -> NOT rejected. Sweep: nsym~3000/prom 23-73 -> untouc
    +     hed.
153 +     if nsym < _MIN_SYMS or (prom < _PROM_MIN and nsym < _NSYM_MIN):
154 +         raise ValueError(f"FSK 버스트가 너무 짧거나 심볼율이 불확실합니다 ({nsym} 심볼)")

91 155     ctr = (np.arange(nsym) + 0.5) * sps
92 156
93 157     # symbol sampling phase: the offset whose sampled levels separate best
94 158     v, best = fcen[:nsym], -1.0
95 159     for o in np.linspace(0, sps, 8, endpoint=False):
96 160         s = fcen[np.clip((ctr + o).astype(int), 0, fcen.size - 1)]
97 161         sep = _sep(s)
98 162         if sep > best:
99 163             best, v = sep, s
100 164
101 165     # level count via spread-collapse (k=2 vs k=4), then cluster

```

```

102 166 k = 4 if _kmeans1d(v, 2)[2] / (_kmeans1d(v, 4)[2] + 1e-9) > _K4_RATIO else 2
103 167 c, lab, spread = _kmeans1d(v, k)
104 168 order = np.argsort(c)
105 169 ranks = np.argsort(order)[lab] # per-symbol level index, 0 = lowest freq
106 170 csort = c[order]
107 171 mid = float(csort.mean()) # debias center: symmetric levels sum ~0
108 172 fc, csort, v = fc + mid, csort - mid, v - mid
109 173 spacing = float(np.mean(np.diff(csort)))
110 174 h = spacing / baud
111 175
112 176 mod = "msk" if (k == 2 and abs(h - 0.5) < 0.15) else f"fsk{k}"
113 177 _, glabels = fsk_levels(mod)
114 178 bits = to_bits(glabels[ranks], mod).astype(np.uint8)
115 179 lock = float(np.clip((spacing / (spread + 1e-9) - 1.5) / 6.5 * 100, 0, 100))
116 180 return dict(mod=mod, fc=fc, baud=baud, h=h, lock=lock, level_freqs=csort,
117 181           symbols=v / (spacing / 2), bits=bits)

```

signus/chirp.py +22 -13 · 코드북 37쪽 · 새로 쓸 줄 10, 23-25, 60-65, 69-80

```

1 1 """Blind linear-chirp / CSS (LoRa) detection + characterization. A constant-envelope
2 2 signal whose instantaneous frequency ramps linearly de-chirps to a tone: the delay-
3 3 conjugate  $x[t] \cdot \text{conj}(x[t-\tau])$  is a beat tone at  $\mu \cdot \tau$  (NONZERO), unlike FSK/CW (beat at
4 4 DC), PSK/QAM (not constant-envelope), or FM-voice/noise (broadband). Characterize-only --
5 5 reported like analog/tone, NEVER demodulated: blind CSS symbol decode needs preamble
6 6 CF0/ST0 sync plus LoRa's reverse-engineered Gray/interleave/Hamming/whitening framing.
7 7 Calibrated on gen chirps vs every other family: chirp beat-PAR>=936, non-chirp<=264."""
8 8
9 9 import numpy as np
10 + from scipy.ndimage import uniform_filter1d
11 from scipy.signal import welch
12
13 _PAR_MIN = 400.0 # delay-conjugate beat-tone peak-to-mean floor (chirp>=936, non-chirp<=264)
14
15 def _bw99(x: np.ndarray, fs: float) -> float:
16 17 """99%-power occupied bandwidth (a chirp fills its band ~flat). Floor-subtracted so the
17 18 noise pedestal in the channel's LPF passband does not inflate the width."""
18 19 f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
19 20 f, p = np.fft.fftshift(f), np.fft.fftshift(p)
20 21 p = np.maximum(p - np.median(p), 0.0) # remove the noise pedestal
21 22 cs = np.cumsum(p) / (p.sum() + 1e-30)
23 + lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)] # clamp BOTH edges: a degenerate PSD
24 + hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)] # (all-zero after floor subtraction)
25 + return float(abs(hi - lo)) # otherwise indexes one past the end
26
27 def _beat(x: np.ndarray, fs: float) -> tuple[float, float]:
28 29 """(peak-to-mean,  $\mu$ [Hz/s]) of the DC-nulled delay-conjugate spectrum."""
29 30 x = x - x.mean()
30 31 tau = max(1, x.size // 200)
31 32 y = x[tau:] * np.conj(x[:-tau])
32 33 nf = 1 << int(np.ceil(np.log2(y.size)))
33 34 spec = np.abs(np.fft.fftshift(np.fft.fft((y - y.mean()) * np.hanning(y.size), nf))) ** 2
34 35 fr = np.fft.fftshift(np.fft.fftfreq(nf, 1 / fs))
35 36 spec[np.abs(fr) < 0.002 * fs] = 0.0 # null DC band: FSK/CW/tone beat here, chirps do not
36 37 k = int(np.argmax(spec))
37 38 return float(spec[k] / (spec.mean() + 1e-30)), float(fr[k] * fs / tau)
38 39
39 40
40 41 def is_chirp(x: np.ndarray, fs: float) -> bool:
41 42 """True when a channel is a linear chirp / CSS (LoRa/FMCW). The beat-tone PAR is the

```

```

42 43     discriminator: a huge margin (chirp>=936 vs PSK/QAM<=94, FSK<=91, FM<=264, noise<=13)
43 44     makes an envelope-CV pre-gate redundant AND harmful (it rejects noisy-but-clear chirps)."""
44 45     if x.size < 256:
45 46         return False
46 47     return _beat(x, fs)[0] >= _PAR_MIN
47 48
48 49
49 50     _SWEEP_PM_MAX = 1.45    # instantaneous-freq histogram peak-to-mean: a band-sweep stays ~1
50 51     _SWEEP_OCC_MIN = 0.6    # ...and fills its band; FSK sits on 2-4 discrete tones (peaky, sparse)
51 52
52 53
53 54 def sweeps_band(x: np.ndarray, fs: float, bins: int = 40) -> bool:
54 55     """True when the instantaneous frequency SWEEPS the channel (linear chirp / CSS) rather
55 56     than hopping between a few discrete FSK tones. A linear FMCW ramp reads bimodal to
56 57     fsk_gate AND trips is_chirp's beat test, so triage needs this to tell a genuine band-sweep
57 58     from FSK: a sweep's IF histogram is flat and fully occupied; FSK's spikes at its tones.
58 59     Calibrated on extracted channels: 0/222 FSK/MSK (snr 8-40) pass, 214/216 FMCW + 60/60 LoRa
60 +     pass (misses only the narrowest, gentlest ramps -- which stay 'fsk', never a regression).
61 +     BOTH the raw and a lightly-smoothed IF must look like a sweep: at moderate SNR (~14) noise
62 +     flattens integer-h FSK's raw histogram to sweep-like pm ~1.44 (knife-edge), but smoothing
63 +     restores its tone spikes (pm 2.5+) while a genuine ramp stays flat (pm ~1.1) -- and at LOW
64 +     SNR smoothing can flatten FSK instead, which the raw view still catches. The conjunction
65 +     only ever narrows 'sweep', so every calibrated chirp above keeps its label (margin >=0.3)."""
66 +     "
60 66     if x.size < 256:
61 67         return False
62 68     x = x - x.mean()
69 +     f0 = np.diff(np.unwrap(np.angle(x))) * fs / (2 * np.pi)
70 +     for f in (f0, uniform_filter1d(f0, 4)):
71 +         lo, hi = np.percentile(f, 1), np.percentile(f, 99)
72 +         if hi - lo < 1:
73 +             return False
74 +         fv = f[(f >= lo) & (f <= hi)]
75 +         h = np.histogram(fv, bins=bins)[0] / fv.size
76 +         pm = h.max() / (h.mean() + 1e-30)    # discrete tones spike this; a sweep ~1
77 +         occ = float((h > 0.005).mean())      # a sweep fills the band; tones occupy few b
78 +         ins
79 +         if not (pm < _SWEEP_PM_MAX and occ >= _SWEEP_OCC_MIN):
80 +             return False
81 +     return True
72 81
73 82
74 83 def analyze_chirp(x: np.ndarray, fs: float) -> dict:
75 84     """Characterize a chirp: {mu[Hz/s], up, bw, par, sf, rs, tsym}. A LoRa hypothesis
76 85     (sf, symbol rate, symbol time) is filled when bw^2/|mu| snaps to 2^(7..12); otherwise
77 86     it stays a generic linear chirp / FMCW. bw is measured on the channel, not asserted."""
78 87     par, mu = _beat(x, fs)
79 88     bw = _bw99(x, fs)
80 89     sf = rs = tsym = None
81 90     if mu != 0 and bw > 0:
82 91         r = float(np.log2(bw * bw / abs(mu)))
83 92         if 7 <= round(r) <= 12 and abs(r - round(r)) < 0.3:    # power-of-two chip count -> LoRa
84 93             sf = int(round(r))
85 94             rs = abs(mu) / bw
86 95             tsym = 2 ** sf / bw
87 96     return {"mu": mu, "up": bool(mu > 0), "bw": float(bw),
88 97            "par": par, "sf": sf, "rs": rs, "tsym": tsym}

```

signus/pipeline.py +65 -5 · 코드북 39쪽 · 새로 쓸 줄 17, 30-33, 37-51, 194-204, 209-213, 219-228, 285-286, 340-348, 351-352, 356-361

```

1 1  """End-to-end blind demodulation. Two families:
2 2     linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
3 3         classify -> fine sync -> [equalizer rescue] -> quality
4 4     fsk (CPFSK/MSK): frequency discriminator (gated by constant envelope)
5 5  Classification runs BEFORE fine sync so the decision-directed loop knows the
6 6  constellation. Differential demapping is an option, not a detection: dqpsk and
7 7  qpsk share a constellation."""
8 8
9 9  import json
10 10 from dataclasses import dataclass, field
11 11
12 12 import numpy as np
13 13
14 14 from . import classify as cl
15 15 from . import dsp, triage
16 16 from .channelize import extract
17 + from .chirp import analyze_chirp, is_chirp, sweeps_band
18 18 from .constellations import demap_bits, demap_diff_bits, mod_order
19 19 from .detect import Detection, detect
20 20 from .eq import equalize, equalize_fse
21 21 from .fsk import analyze_fsk, fsk_gate
22 22 from .sigio import Meta, parse_name, read
23 23 from .spectrum import spectrum, waterfall
24 24 from .sync import find_preamble
25 25
26 26 _BITS_CAP = 65536
27 27 _EQ_LOCK = 60.0    # below this a linear signal is worth an equalizer attempt
28 28 _EQ_GAIN = 3.0     # ...keep it only if lock improves by this much
29 29 _EQ_FLOOR = 50.0   # ...and clears this floor, so a wrong-mod fit is never locked in
30 + _EQ_QAM_FLOOR = 60.0 # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop c
  +   an
31 + #                               drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51)
  +   , a
32 + #                               confident WRONG order. Genuine multipath recoveries clear ~72; the sweep
  +   's
33 + #                               eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identic
  +   al.
30 34 _BAUD_GAIN = 5.0    # an in-band baud hypothesis must beat the global one by this lock
31 35 _SHORT_LOCK = 60.0 # below this, retry the baud line with fewer blocks (short-burst rescue)
32 36 _SYNC_LOCK = 60.0  # below this, try a repeated-preamble carrier/timing sync (packetized bursts
  +   )
37 + _ALIAS_MARGIN = 10.0 # a preamble carrier alias must beat the next by this lock, else ambiguous
  +   .
38 + #                               This is the LOAD-BEARING rotation guard: even when the coarse anchor itse
  +   lf
39 + #                               aliases and validates a rotation at ~80 lock, that rotation wins by a thi
  +   n
40 + #                               margin (~8) -- so a 10-lock margin refuses it. Because the margin catche
  +   s it,
41 + #                               the lock floor can stay modest (78, not 82) -> ~5% more genuine recoverie
  +   s.
42 + _SYNC_ACCEPT = 78.0 # ...AND clear this absolute lock: below it correct/rotated aliases overlap
  +   . A
43 + #                               65 floor was tried (to recover more moderate bursts) but an independent
44 + #                               snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-7
  +   8 band
45 + #                               (carrier off by one baud/L step -> confident garbage bits). Even 78 leak
  +   s a
46 + #                               a few hard-corner rotations (a separate pre-existing gate limit): hold a

```

```

47 + _SYNC_ADIST = 1.5      # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
48 + #                      soft backstop to the margin: it catches the one rotation the margin misse
49 + #                      but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
50 + #                      large-offset recoveries. The three together give 0 confident-wrong across
51 + #                      1920 hard-corner bursts at the max recall this gate reaches.
34 52 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"} # pi4dqpsk arrives as qpsk -> dqpsk
35 53
36 54
37 55 @dataclass
38 56 class Result:
39 57     meta: Meta
40 58     family: str                # 'linear' | 'fsk'
41 59     burst: tuple[int, int]
42 60     fc: float
43 61     baud: float
44 62     mod: str
45 63     lock: float
46 64     symbols: np.ndarray        # aligned symbols (linear) / normalized levels (fsk)
47 65     bits: np.ndarray
48 66     iq_corr: np.ndarray        # exportable corrected baseband
49 67     burst_x: np.ndarray = field(repr=False, default=None) # for spectrum views
50 68     symmetry: int = 0
51 69     carrier_ambiguous: bool = False
52 70     baud_conf: float = 0.0
53 71     rolloff: float | None = None
54 72     h: float | None = None     # FSK modulation index
55 73     evm: float = float("nan")
56 74     mer_db: float = float("nan")
57 75     occupied: int = 0
58 76     snr_est: float = float("nan")
59 77     eq_applied: bool = False
60 78     eq_mode: str | None = None # 'sym' | 'fse'
61 79     alias_resolved: bool = False
62 80     baud_fallback: bool = False
63 81     bursts: list = field(default_factory=list)
64 82     burst_idx: int = 0
65 83     preamble: object = None   # sync.Preamble when a repeated preamble drove the decode
66 84
67 85     def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
68 86         z = np.nan_to_num(self.symbols) # a dead/DC capture -> NaN symbols -> invalid JSON
69 87         if z.size > max_points: # keep TIME ORDER: the UI animates this sequence
70 88             z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
71 89         det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
72 90             "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
73 91             "baud_conf": round(self.baud_conf, 1),
74 92             "alias_resolved": self.alias_resolved,
75 93             "baud_fallback": self.baud_fallback}
76 94         det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
77 95         rf = self.meta.rf_center # real RF = capture centre + baseband fc
78 96         det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
79 97         if self.h is not None:
80 98             det["h"] = round(self.h, 3)
81 99         if self.preamble is not None: # a repeated preamble drove the sync
82 100             p = self.preamble
83 101             det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
84 102                 "conf": round(p.conf, 2), "start": p.start, "end": p.end}
85 103         doc = {
86 104             "fs": self.meta.fs, "fmt": self.meta.fmt, "rf_center": self.meta.rf_center,

```

```

87 105         "family": self.family, "n_samples": int(self.iq_corr.size),
88 106         "burst": {"start": self.burst[0], "end": self.burst[1]},
89 107         "bursts": [{"start": s, "end": e} for s, e in self.bursts],
90 108         "burst_idx": self.burst_idx,
91 109         "detected": det,
92 110         "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
93 111                    "evm": _r(self.evm, 4)},
94 112         "snr_est_db": _r(self.snr_est),
95 113         "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
96 114         "constellation": {"i": np.round(z.real, 4).tolist(),
97 115                           "q": np.round(z.imag, 4).tolist()},
98 116         "bits": "".join(map(str, self.bits[:_BITS_CAP])),
99 117     }
100 118     if views and self.burst_x is not None:
101 119         doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
102 120         doc["waterfall"] = waterfall(self.burst_x, self.meta.fs)
103 121     return doc
104 122
105 123     def save_iq(self, path: str) -> None:
106 124         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
107 125
108 126     def save_symbols(self, path: str) -> None:
109 127         np.save(path, self.symbols.astype(np.complex64))
110 128
111 129     def save_bits(self, path: str, packed: bool = False) -> None:
112 130         if packed:
113 131             np.packbits(self.bits).tofile(path)
114 132         else:
115 133             with open(path, "w") as fh:
116 134                 fh.write("".join(map(str, self.bits)))
117 135
118 136     def save_report(self, path: str) -> None:
119 137         with open(path, "w") as fh:
120 138             json.dump(self.to_json(), fh, indent=1)
121 139
122 140
123 141     def _r(v: float, nd: int = 2) -> float | None:
124 142         return None if v is None or not np.isfinite(v) else round(float(v), nd)
125 143
126 144
127 145     def _fine(z: np.ndarray, mod: str) -> np.ndarray:
128 146         """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
129 147         return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
130 148
131 149
132 150     def _blocks(n: int) -> int:
133 151         """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
134 152         low SNR) wants fewer, longer segments; only long records need more for
135 153         stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
136 154         return int(np.clip(n // 30000, 1, 8))
137 155
138 156
139 157     def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
140 158         """Matched filter + timing + classify + fine sync at one baud hypothesis."""
141 159         alpha = dsp.est_rolloff(xd, fs, baud)
142 160         ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
143 161         raw = dsp.timing(ym, 4)
144 162         mod = cl.classify(raw, symmetry)
145 163         # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
146 164         # the bulk phase first kills the loop transient that would corrupt the first symbols

```

```

147 165     syms = _fine(raw, mod)
148 166     return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
149 167
150 168
151 169 def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
152 170     """Equalize with a seed constellation, then let the OPENED eye pick the mod:
153 171     heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
154 172     with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
155 173     z1 = eq_fn(z, seed_mod)
156 174     cand_s = []
157 175     for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}: # symmetry too, per the docstrin
158 176         s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
159 177         cand_s.append((cl.quality(s, m), s, m))
160 178     best = max(cand_s, key=lambda c: c[0].lock)
161 179     # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
162 180     # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
163 181     # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
164 182     # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
165 183     # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
166 184     lo = [c for c in cand_s if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
167 185     if lo and best[2] == "qpsk":
168 186         return lo[0]
169 187     return best
170 188
171 189
172 190 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
173 191           burst: int | None = None) -> Result:
174 192     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
175 193     `burst` selects one detected burst (default: the most energetic)."""
176 194 +     if x.size:
177 195 +         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
178 196 +         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
179 197 +         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
180 198 +         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
181 199 +         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
182 200 +         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
183 201 +         scale = float(np.percentile(np.abs(x[:16]), 99)) # strided: scale detection needs the
184 202 +         # BULK magnitude, not every sample -- 16x cheaper on a large direct capture
185 203 +         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
186 204 +             x = x / scale
187 205
188 206     x = dsp.analytic(x)
189 207     x = x - x.mean() # DC block: LO leakage otherwise corrupts every estimator
190 208     if x.size < 64: # empty / trivially short: nothing to estimate, and it crashes downstream
191 209         raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
192 210 +     rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
193 211 +     if rms and (rms < 1e-6 or rms > 1e6): # only PATHOLOGICAL scale: normalize so the scale-var
194 212 +         # iant
195 213 +         x = x / rms # spots (fsk_gate's CV epsilon, est_carrier's x**p ov
196 214 +         # er/
197 215 +         # underflow) stay invariant. Normal captures rms~0(1
198 216 +         # ) ->
199 217 +         # untouched -> the acceptance sweep is byte-identical
200 218 +         .
201 219
202 220     bursts = dsp.find_bursts(x, meta.fs)
203 221     if burst is None or not 0 <= burst < len(bursts):
204 222         burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
205 223     s, e = bursts[burst]
206 224     xb = x[s:e] if e - s >= 64 else x
207 225
208 226
209 227
210 228
211 229
212 230
213 231
214 232
215 233
216 234
217 235
218 236
219 237
220 238
221 239
222 240
223 241
224 242
225 243
226 244
227 245
228 246
229 247
230 248
231 249
232 250
233 251
234 252
235 253
236 254
237 255
238 256
239 257
240 258
241 259
242 260
243 261
244 262
245 263
246 264
247 265
248 266
249 267
250 268
251 269
252 270
253 271
254 272
255 273
256 274
257 275
258 276
259 277
260 278
261 279
262 280
263 281
264 282
265 283
266 284
267 285
268 286
269 287
270 288
271 289
272 290
273 291
274 292
275 293
276 294
277 295
278 296
279 297
280 298
281 299
282 300
283 301
284 302
285 303
286 304
287 305
288 306
289 307
290 308
291 309
292 310
293 311
294 312
295 313
296 314
297 315
298 316
299 317
300 318
301 319
302 320
303 321
304 322
305 323
306 324
307 325
308 326
309 327
310 328
311 329
312 330
313 331
314 332
315 333
316 334
317 335
318 336
319 337
320 338
321 339
322 340
323 341
324 342
325 343
326 344
327 345
328 346
329 347
330 348
331 349
332 350
333 351
334 352
335 353
336 354
337 355
338 356
339 357
340 358
341 359
342 360
343 361
344 362
345 363
346 364
347 365
348 366
349 367
350 368
351 369
352 370
353 371
354 372
355 373
356 374
357 375
358 376
359 377
360 378
361 379
362 380
363 381
364 382
365 383
366 384
367 385
368 386
369 387
370 388
371 389
372 390
373 391
374 392
375 393
376 394
377 395
378 396
379 397
380 398
381 399
382 400
383 401
384 402
385 403
386 404
387 405
388 406
389 407
390 408
391 409
392 410
393 411
394 412
395 413
396 414
397 415
398 416
399 417
400 418
401 419
402 420
403 421
404 422
405 423
406 424
407 425
408 426
409 427
410 428
411 429
412 430
413 431
414 432
415 433
416 434
417 435
418 436
419 437
420 438
421 439
422 440
423 441
424 442
425 443
426 444
427 445
428 446
429 447
430 448
431 449
432 450
433 451
434 452
435 453
436 454
437 455
438 456
439 457
440 458
441 459
442 460
443 461
444 462
445 463
446 464
447 465
448 466
449 467
450 468
451 469
452 470
453 471
454 472
455 473
456 474
457 475
458 476
459 477
460 478
461 479
462 480
463 481
464 482
465 483
466 484
467 485
468 486
469 487
470 488
471 489
472 490
473 491
474 492
475 493
476 494
477 495
478 496
479 497
480 498
481 499
482 500
483 501
484 502
485 503
486 504
487 505
488 506
489 507
490 508
491 509
492 510
493 511
494 512
495 513
496 514
497 515
498 516
499 517
500 518
501 519
502 520
503 521
504 522
505 523
506 524
507 525
508 526
509 527
510 528
511 529
512 530
513 531
514 532
515 533
516 534
517 535
518 536
519 537
520 538
521 539
522 540
523 541
524 542
525 543
526 544
527 545
528 546
529 547
530 548
531 549
532 550
533 551
534 552
535 553
536 554
537 555
538 556
539 557
540 558
541 559
542 560
543 561
544 562
545 563
546 564
547 565
548 566
549 567
550 568
551 569
552 570
553 571
554 572
555 573
556 574
557 575
558 576
559 577
560 578
561 579
562 580
563 581
564 582
565 583
566 584
567 585
568 586
569 587
570 588
571 589
572 590
573 591
574 592
575 593
576 594
577 595
578 596
579 597
580 598
581 599
582 600
583 601
584 602
585 603
586 604
587 605
588 606
589 607
590 608
591 609
592 610
593 611
594 612
595 613
596 614
597 615
598 616
599 617
600 618
601 619
602 620
603 621
604 622
605 623
606 624
607 625
608 626
609 627
610 628
611 629
612 630
613 631
614 632
615 633
616 634
617 635
618 636
619 637
620 638
621 639
622 640
623 641
624 642
625 643
626 644
627 645
628 646
629 647
630 648
631 649
632 650
633 651
634 652
635 653
636 654
637 655
638 656
639 657
640 658
641 659
642 660
643 661
644 662
645 663
646 664
647 665
648 666
649 667
650 668
651 669
652 670
653 671
654 672
655 673
656 674
657 675
658 676
659 677
660 678
661 679
662 680
663 681
664 682
665 683
666 684
667 685
668 686
669 687
670 688
671 689
672 690
673 691
674 692
675 693
676 694
677 695
678 696
679 697
680 698
681 699
682 700
683 701
684 702
685 703
686 704
687 705
688 706
689 707
690 708
691 709
692 710
693 711
694 712
695 713
696 714
697 715
698 716
699 717
700 718
701 719
702 720
703 721
704 722
705 723
706 724
707 725
708 726
709 727
710 728
711 729
712 730
713 731
714 732
715 733
716 734
717 735
718 736
719 737
720 738
721 739
722 740
723 741
724 742
725 743
726 744
727 745
728 746
729 747
730 748
731 749
732 750
733 751
734 752
735 753
736 754
737 755
738 756
739 757
740 758
741 759
742 760
743 761
744 762
745 763
746 764
747 765
748 766
749 767
750 768
751 769
752 770
753 771
754 772
755 773
756 774
757 775
758 776
759 777
760 778
761 779
762 780
763 781
764 782
765 783
766 784
767 785
768 786
769 787
770 788
771 789
772 790
773 791
774 792
775 793
776 794
777 795
778 796
779 797
780 798
781 799
782 800
783 801
784 802
785 803
786 804
787 805
788 806
789 807
790 808
791 809
792 810
793 811
794 812
795 813
796 814
797 815
798 816
799 817
800 818
801 819
802 820
803 821
804 822
805 823
806 824
807 825
808 826
809 827
810 828
811 829
812 830
813 831
814 832
815 833
816 834
817 835
818 836
819 837
820 838
821 839
822 840
823 841
824 842
825 843
826 844
827 845
828 846
829 847
830 848
831 849
832 850
833 851
834 852
835 853
836 854
837 855
838 856
839 857
840 858
841 859
842 860
843 861
844 862
845 863
846 864
847 865
848 866
849 867
850 868
851 869
852 870
853 871
854 872
855 873
856 874
857 875
858 876
859 877
860 878
861 879
862 880
863 881
864 882
865 883
866 884
867 885
868 886
869 887
870 888
871 889
872 890
873 891
874 892
875 893
876 894
877 895
878 896
879 897
880 898
881 899
882 900
883 901
884 902
885 903
886 904
887 905
888 906
889 907
890 908
891 909
892 910
893 911
894 912
895 913
896 914
897 915
898 916
899 917
900 918
901 919
902 920
903 921
904 922
905 923
906 924
907 925
908 926
909 927
910 928
911 929
912 930
913 931
914 932
915 933
916 934
917 935
918 936
919 937
920 938
921 939
922 940
923 941
924 942
925 943
926 944
927 945
928 946
929 947
930 948
931 949
932 950
933 951
934 952
935 953
936 954
937 955
938 956
939 957
940 958
941 959
942 960
943 961
944 962
945 963
946 964
947 965
948 966
949 967
950 968
951 969
952 970
953 971
954 972
955 973
956 974
957 975
958 976
959 977
960 978
961 979
962 980
963 981
964 982
965 983
966 984
967 985
968 986
969 987
970 988
971 989
972 990
973 991
974 992
975 993
976 994
977 995
978 996
979 997
980 998
981 999
982 1000
983 1001
984 1002
985 1003
986 1004
987 1005
988 1006
989 1007
990 1008
991 1009
992 1010
993 1011
994 1012
995 1013
996 1014
997 1015
998 1016
999 1017
1000 1018
1001 1019
1002 1020
1003 1021
1004 1022
1005 1023
1006 1024
1007 1025
1008 1026
1009 1027
1010 1028
1011 1029
1012 1030
1013 1031
1014 1032
1015 1033
1016 1034
1017 1035
1018 1036
1019 1037
1020 1038
1021 1039
1022 1040
1023 1041
1024 1042
1025 1043
1026 1044
1027 1045
1028 1046
1029 1047
1030 1048
1031 1049
1032 1050
1033 1051
1034 1052
1035 1053
1036 1054
1037 1055
1038 1056
1039 1057
1040 1058
1041 1059
1042 1060
1043 1061
1044 1062
1045 1063
1046 1064
1047 1065
1048 1066
1049 1067
1050 1068
1051 1069
1052 1070
1053 1071
1054 1072
1055 1073
1056 1074
1057 1075
1058 1076
1059 1077
1060 1078
1061 1079
1062 1080
1063 1081
1064 1082
1065 1083
1066 1084
1067 1085
1068 1086
1069 1087
1070 1088
1071 1089
1072 1090
1073 1091
1074 1092
1075 1093
1076 1094
1077 1095
1078 1096
1079 1097
1080 1098
1081 1099
1082 1100
1083 1101
1084 1102
1085 1103
1086 1104
1087 1105
1088 1106
1089 1107
1090 1108
1091 1109
1092 1110
1093 1111
1094 1112
1095 1113
1096 1114
1097 1115
1098 1116
1099 1117
1100 1118
1101 1119
1102 1120
1103 1121
1104 1122
1105 1123
1106 1124
1107 1125
1108 1126
1109 1127
1110 1128
1111 1129
1112 1130
1113 1131
1114 1132
1115 1133
1116 1134
1117 1135
1118 1136
1119 1137
1120 1138
1121 1139
1122 1140
1123 1141
1124 1142
1125 1143
1126 1144
1127 1145
1128 1146
1129 1147
1130 1148
1131 1149
1132 1150
1133 1151
1134 1152
1135 1153
1136 1154
1137 1155
1138 1156
1139 1157
1140 1158
1141 1159
1142 1160
1143 1161
1144 1162
1145 1163
1146 1164
1147 1165
1148 1166
1149 1167
1150 1168
1151 1169
1152 1170
1153 1171
1154 1172
1155 1173
1156 1174
1157 1175
1158 1176
1159 1177
1160 1178
1161 1179
1162 1180
1163 1181
1164 1182
1165 1183
1166 1184
1167 1185
1168 1186
1169 1187
1170 1188
1171 1189
1172 1190
1173 1191
1174 1192
1175 1193
1176 1194
1177 1195
1178 1196
1179 1197
1180 1198
1181 1199
1182 1200
1183 1201
1184 1202
1185 1203
1186 1204
1187 1205
1188 1206
1189 1207
1190 1208
1191 1209
1192 1210
1193 1211
1194 1212
1195 1213
1196 1214
1197 1215
1198 1216
1199 1217
1200 1218
1201 1219
1202 1220
1203 1221
1204 1222
1205 1223
1206 1224
1207 1225
1208 1226
1209 1227
1210 1228
1211 1229
1212 1230
1213 1231
1214 1232
1215 1233
1216 1234
1217 1235
1218 1236
1219 1237
1220 1238
1221 1239
1222 1240
1223 1241
1224 1242
1225 1243
1226 1244
1227 1245
1228 1246
1229 1247
1230 1248
1231 1249
1232 1250
1233 1251
1234 1252
1235 1253
1236 1254
1237 1255
1238 1256
1239 1257
1240 1258
1241 1259
1242 1260
1243 1261
1244 1262
1245 1263
1246 1264
1247 1265
1248 1266
1249 1267
1250 1268
1251 1269
1252 1270
1253 1271
1254 1272
1255 1273
1256 1274
1257 1275
1258 1276
1259 1277
1260 1278
1261 1279
1262 1280
1263 1281
1264 1282
1265 1283
1266 1284
1267 1285
1268 1286
1269 1287
1270 1288
1271 1289
1272 1290
1273 1291
1274 1292
1275 1293
1276 1294
1277 1295
1278 1296
1279 1297
1280 1298
1281 1299
1282 1300
1283 1301
1284 1302
1285 1303
1286 1304
1287 1305
1288 1306
1289 1307
1290 1308
1291 1309
1292 1310
1293 1311
1294 1312
1295 1313
1296 1314
1297 1315
1298 1316
1299 1317
1300 1318
1301 1319
1302 1320
1303 1321
1304 1322
1305 1323
1306 1324
1307 1325
1308 1326
1309 1327
1310 1328
1311 1329
1312 1330
1313 1331
1314 1332
1315 1333
1316 1334
1317 1335
1318 1336
1319 1337
1320 1338
1321 1339
1322 1340
1323 1341
1324 1342
1325 1343
1326 1344
1327 1345
1328 1346
1329 1347
1330 1348
1331 1349
1332 1350
1333 1351
1334 1352
1335 1353
1336 1354
1337 1355
1338 1356
1339 1357
1340 1358
1341 1359
1342 1360
1343 1361
1344 1362
1345 1363
1346 1364
1347 1365
1348 1366
1349 1367
1350 1368
1351 1369
1352 1370
1353 1371
1354 1372
1355 1373
1356 1374
1357 1375
1358 1376
1359 1377
1360 1378
1361 1379
1362 1380
1363 1381
1364 1382
1365 1383
1366 1384
1367 1385
1368 1386
1369 1387
1370 1388
1371 1389
1372 1390
1373 1391
1374 1392
1375 1393
1376 1394
1377 1395
1378 1396
1379 1397
1380 1398
1381 1399
1382 1400
1383 1401
1384 1402
1385 1403
1386 1404
1387 1405
1388 1406
1389 1407
1390 1408
1391 1409
1392 1410
1393 1411
1394 1412
1395 1413
1396 1414
1397 1415
1398 1416
1399 1417
1400 1418
1401 1419
1402 1420
1403 1421
1404 1422
1405 1423
1406 1424
1407 1425
1408 1426
1409 1427
1410 1428
1411 1429
1412 1430
1413 1431
1414 1432
1415 1433
1416 1434
1417 1435
1418 1436
1419 1437
1420 1438
1421 1439
1422 1440
1423 1441
1424 1442
1425 1443
1426 1444
1427 1445
1428 1446
1429 1447
1430 1448
1431 1449
1432 1450
1433 1451
1434 1452
1435 1453
1436 1454
1437 1455
1438 1456
1439 1457
14
```

```

220 + # chirp gate -- same conjunctive tie-break as triage.family, so direct analyze / survey_web
221 + # single mode agree with survey's label: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and
222 + # would force-demodulate into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass)
223 + # out of this branch, so only a genuine band-sweep is refused -- never demodulated.
224 + if is_chirp(xb, meta.fs) and sweeps_band(xb, meta.fs):
225 +     info = analyze_chirp(xb, meta.fs)
226 +     kind = f"LoRa 추정 SF{info['sf']}" if info["sf"] else f"μ={info['mu']:.3g} Hz/s"
227 +     raise ValueError(f"처프(FMCW/CSS) 신호입니다 ({kind}, bw≈{info['bw']:.3g} Hz) - "
228 +                     "정상도 복조 대상이 아닙니다 (서베이 모드에서 특성 보고)")

185 229
186 230 if fsk_gate(xb, meta.fs):
187 231     r = analyze_fsk(xb, meta.fs)
188 232     sym = np.asarray(r["symbols"], dtype=np.complex128)
189 233     return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
190 234                          sym, r["bits"], xb, burst_x=xb, h=r["h"],
191 235                          bursts=bursts, burst_idx=burst)
192 236
193 237 fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
194 238 fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
195 239 amb = abs(symmetry * fc) > 0.4 * meta.fs
196 240 xd = dsp.mix(xb, meta.fs, fc)
197 241
198 242 baud, conf = dsp.est_baud(xd, meta.fs)
199 243 fell = False
200 244 alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
201 245 bw = dsp.occupied_bw(xd, meta.fs)
202 246 if bw and not (0.72 * bw <= baud <= 1.05 * bw):
203 247     # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
204 248     # (and under harsh channels) data junk outruns the true line. Demod the
205 249     # in-band hypotheses too and let lock decide, with a clear margin.
206 250     for lo in (0.80, 0.67):
207 251         b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
208 252         if abs(b2 - baud) <= 0.01 * b2:
209 253             continue
210 254         t = _demod(xd, meta.fs, b2, symmetry)
211 255         if t[5].lock > q.lock + _BAUD_GAIN:
212 256             alpha, ym, raw, mod, syms, q = t
213 257             baud, conf, fell = b2, c2, True
214 258
215 259 if q.lock < _SHORT_LOCK:
216 260     # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
217 261     # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
218 262     # carrier_blocks logic, applied to baud). Run both and keep only a clearly-better
219 263     # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
220 264     for nb in (2, 1):
221 265         b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
222 266         if abs(b2 - baud) <= 0.01 * b2:
223 267             continue
224 268         t = _demod(xd, meta.fs, b2, symmetry)
225 269         if t[5].lock > q.lock + _BAUD_GAIN:
226 270             alpha, ym, raw, mod, syms, q = t
227 271             baud, conf, fell = b2, c2, True
228 272
229 273 eq_applied, eq_mode, pre = False, None, None
230 274 if q.lock < _EQ_LOCK: # ISI (multipath) is what a phase-only loop cannot fix
231 275     # CMA is modulus-only, so it opens the eye without knowing the constellation.
232 276     qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
233 277     mode = "sym"
234 278     if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
235 279         # symbol-spaced FIR could not invert the channel; retry T/2-spaced on

```

```

236 280         # the clock-tracked 2 sps stream (unaliaised spectrum, longer inverse)
237 281         q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
238 282                             dsp.timing(ym, 4, out=2), mod, symmetry)
239 283         if q2.lock > qe.lock:
240 284             qe, se, me, mode = q2, s2, m2, "fse"
285 +         floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
286 +         if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
242 287             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
243 288     elif mod in ("16qam", "32qam", "64qam"):
244 289         # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
245 290         # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
246 291         # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
247 292         # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched
248 293         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
249 294         mode = "sym"
250 295         if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
251 296             # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
252 297             # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
253 298             # QAM (no lower order) never pays for the FSE.
254 299             qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
255 300                                 dsp.timing(ym, 4, out=2), mod, symmetry)
256 301             mode = "fse"
257 302         if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
258 303             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
259 304
260 305     if q.lock < _SYNC_LOCK:
261 306         # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
262 307         # carrier/baud/timing from only a few symbols, where the blind M-th-power estimator (nee
263 308         # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock
264 309         # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
265 310         # reaches here. Detect the preamble, mix by its CF0, demod the DATA past it, keep-best --
266 311         # random / no-preamble signal yields no plateau or a CF0 that does not improve lock an
267 312         # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the b
268 313         # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-
269 314         # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
270 315         ps = find_preamble(xb, meta.fs, baud_hint=baud)
271 316         if ps is not None:
272 317             # The Schmidl-Cox CF0 is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by bau
273 318             # rotates a whole constellation position per symbol, so the eye still looks locked b
274 319             # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coar
275 320             # M-th-power fc lands within ~1 alias step of the truth even on these short bursts
276 321             # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct a
277 322             # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
278 323             step = meta.fs / ps.period
279 324             k0 = round((fc - ps.cfo_hz) / step)
280 325             cands = []
281 326             for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
282 327                 for k in range(k0 - 3, k0 + 4):

```

```

283 328         cfo = ps.cfo_hz + k * step
284 329         data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
285 330         if data.size < 256:
286 331             continue
287 332         for sm in (2, 4, 8):
288 333             t = _demod(data, meta.fs, float(b2), sm)
289 334             cand.append((t[5].lock, cfo, float(b2), sm, t))
290 335         cand.sort(key=lambda c: -c[0])
291 336         # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG on
292 337         # e by a
293 338         # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear g
294 339         # ap;
295 340         # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rat
296 341         # her
297 342         # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete
298 343         # .)
299 344         # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -
300 345         # - a
301 346         # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alo
302 347         # ne
303 348         # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38)
304 349         # . Two
305 350         # conjunctive conditions favour precision over recall: (1) a strong absolute lock, a
306 351         # nd
307 352         # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -
308 353         # - a
309 354         # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
310 355         # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); o
311 356         # n the
312 357         # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
313 358         if cand and cand[0][0] >= _SYNC_ACCEPT and cand[0][0] > q.lock + _EQ_GAIN:
314 359             top = cand[0]
315 360             other = next((c for c in cand if abs(c[1] - top[1]) > step / 2), None)
316 361             anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
317 362             if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
318 363                 alpha, ym, raw, mod, syms, q = top[4]
319 364                 fc, baud, symmetry = top[1], top[2], top[3]
320 365                 eq_applied, eq_mode, pre = False, None, ps
321 366                 # diagnostics must describe the ACCEPTED decode, not the abandoned blind
322 367                 # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
323 368                 # keeps alias_resolved False), the baud from the preamble period (no
324 369                 # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
325 370                 fc0, conf, fell = fc, 0.0, False
326 371                 amb = abs(symmetry * fc) > 0.4 * meta.fs
327 372
328 373         bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
329 374         else demap_bits(syms, mod)
330 375         return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
331 376         burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
332 377         rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
333 378         snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
334 379         alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
335 380         bursts=bursts, burst_idx=burst, preamble=pre)
336 381
337 382 def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
338 383 dtype: str | None = None, endian: str | None = None,
339 384 bitrev: bool | None = None, diff: bool = False,
340 385 burst: int | None = None, rf: float | None = None) -> Result:
341 386 """Analyze a file; explicit args override SigMF/filename tokens."""

```

```

318 378     from .sigio import parse_sigmf
319 379     m = parse_sigmf(path) or parse_name(path)
320 380     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
321 381                endian or m.endian, m.bitrev if bitrev is None else bitrev,
322 382                rf_center=rf if rf is not None else m.rf_center)
323 383     x, meta = read(path, meta)
324 384     return analyze(x, meta, diff, burst)
325 385
326 386
327 387 # --- wideband survey: many emitters in one capture -----
328 388
329 389 @dataclass
330 390 class Emitter:
331 391     detection: Detection
332 392     kind: str                # linear|fsk|chirp|analog|tone|tooshort|error
333 393     abs_fc: float            # emitter carrier vs capture centre (Hz)
334 394     result: Result | None = None # demod result for digital kinds
335 395     info: dict | None = None    # characterization for non-digital kinds (e.g. chirp params)
336 396
337 397     def to_json(self) -> dict:
338 398         d = self.detection
339 399         doc = {"kind": self.kind, "abs_fc": round(self.abs_fc, 3),
340 400              "det": {"fc": round(d.fc, 3), "bw": round(d.bw, 3),
341 401                    "t0": d.t0, "t1": d.t1, "snr_db": _r(d.snr_db),
342 402                    "baud_hint": round(d.baud_hint, 1)}}
343 403         if self.info is not None:
344 404             doc["info"] = self.info
345 405         if self.result is not None:
346 406             r = self.result
347 407             doc.update(mod=r.mod, baud=round(r.baud, 3), lock=round(r.lock, 1),
348 408                      mer_db=_r(r.mer_db), evm=_r(r.evm, 4), family=r.family)
349 409         return doc
350 410
351 411     def to_detail(self, rf_center: float | None = None) -> dict:
352 412         """Web drill-down payload: the box summary plus, for a digital emitter, the
353 413         full analyze result so the UI reuses its single-signal detail view verbatim.
354 414         The channel is demodulated at baseband (no rf_center), and r.fc is only the
355 415         residual within-channel offset -- so the emitter's absolute RF is the capture
356 416         centre plus abs_fc (the full offset from capture centre), injected here."""
357 417         doc = self.to_json()
358 418         if self.result is not None:
359 419             doc["result"] = self.result.to_json()
360 420             if rf_center is not None:
361 421                 doc["result"]["detected"]["rf_hz"] = round(rf_center + self.abs_fc, 3)
362 422         return doc
363 423
364 424
365 425 @dataclass
366 426 class Survey:
367 427     meta: Meta
368 428     emitters: list[Emitter] = field(default_factory=list)
369 429
370 430     def to_json(self) -> dict:
371 431         return {"fs": self.meta.fs, "fmt": self.meta.fmt,
372 432              "n_emitters": len(self.emitters),
373 433              "emitters": [e.to_json() for e in self.emitters]}
374 434
375 435
376 436 def survey(x: np.ndarray, meta: Meta, *, diff: bool = False, **detect_kw) -> Survey:
377 437     """Detect every emitter in a wideband capture, extract each to its own baseband

```

```

378 438         channel, and demodulate the digital ones with the unchanged single-signal chain.
379 439         A capture holding one signal that fills the band collapses to a single emitter."""
380 440     xa = dsp.analytic(x)
381 441     xa = xa - xa.mean()
382 442     emitters = []
383 443     for d in detect(xa, meta.fs, **detect_kw):
384 444         ch, fs_ch = extract(xa, meta.fs, d)
385 445         if ch.size < 256:
386 446             emitters.append(Emitter(d, "tooshort", d.fc))
387 447             continue
388 448         kind = triage.family(ch, fs_ch)
389 449         if kind in ("linear", "fsk"):
390 450             # triage only gates digital vs not; analyze's own family is authoritative.
391 451             # Isolate each channel: a degenerate one must not abort the whole survey.
392 452             try:
393 453                 r = analyze(ch, Meta(fs_ch, "iq", "f32", "le", False), diff=diff)
394 454                 emitters.append(Emitter(d, r.family, d.fc + r.fc, r))
395 455             except Exception:
396 456                 emitters.append(Emitter(d, "error", d.fc))
397 457         else:
398 458             info = analyze_chirp(ch, fs_ch) if kind == "chirp" else None
399 459             emitters.append(Emitter(d, kind, d.fc, info=info))
400 460     return Survey(meta, emitters)
401 461
402 462
403 463 def survey_web(x: np.ndarray, meta: Meta, *, diff: bool = False) -> dict:
404 464     """Web survey payload for /api/survey. When detect sees <=1 emitter the capture
405 465     is effectively one signal -> 'single' mode returns the UNCHANGED direct analyze()
406 466     result (correct fc, matches /api/analyze). Otherwise 'survey' mode adds a whole-
407 467     capture overview waterfall and every emitter's drill-down detail. Additive: does
408 468     not alter survey()/Survey/Emitter, so CLI + existing tests are unaffected."""
409 469     xa = dsp.analytic(x)
410 470     xa = xa - xa.mean()
411 471     if len(detect(xa, meta.fs)) <= 1:
412 472         r = analyze(x, meta, diff=diff)
413 473         out = {"mode": "single", "result": r.to_json()}
414 474         if len(r.bursts) > 1: # multi-burst signal -> whole-record map for burst selection
415 475             out["overview"] = {"n": int(xa.size), "fs": meta.fs,
416 476                               "waterfall": waterfall(xa, meta.fs)}
417 477         return out
418 478     sv = survey(x, meta, diff=diff)
419 479     return {"mode": "survey", "fs": meta.fs, "fmt": meta.fmt, "rf_center": meta.rf_center,
420 480           "overview": {"fs": meta.fs, "n": int(xa.size), "spectrum": spectrum(xa, meta.fs),
421 481                       "waterfall": waterfall(xa, meta.fs)},
422 482           "emitters": [e.to_detail(meta.rf_center) for e in sv.emitters]}
423 483
424 484
425 485 def survey_file(path: str, fs: float | None = None, fmt: str | None = None,
426 486                dtype: str | None = None, endian: str | None = None,
427 487                bitrev: bool | None = None, diff: bool = False,
428 488                rf: float | None = None) -> Survey:
429 489     """Survey a capture file; explicit args override SigMF/filename tokens."""
430 490     from .sigio import parse_sigmf
431 491     m = parse_sigmf(path) or parse_name(path)
432 492     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
433 493               endian or m.endian, m.bitrev if bitrev is None else bitrev,
434 494               rf_center=rf if rf is not None else m.rf_center)
435 495     x, meta = read(path, meta)
436 496     return survey(x, meta, diff=diff)

```

signus/server.py +1 -1 · 코드북 55쪽 · 새로 쓸 줄 12

```

1 1 """Stdlib-only web server: static frontend + POST /api/analyze (raw body,
2 2 filename + optional fs/fmt/dtype overrides in the query string - no multipart)."""
3 3
4 4 import json
5 5 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
6 6 from pathlib import Path
7 7 from urllib.parse import parse_qs, urlparse
8 8
9 9 from .pipeline import analyze, survey_web
10 10 from .sigio import Meta, decode, parse_name
11 11
12 + _WEB = Path(__file__).resolve().parent / "web" # inside the package so wheels/sdists ship it
13 13 _MIME = {"html": "text/html", "css": "text/css", "js": "text/javascript",
14 14         "json": "application/json", "png": "image/png", "svg": "image/svg+xml"}
15 15 _MAX_BODY = 256 * 1024 * 1024
16 16
17 17
18 18 class Handler(BaseHTTPRequestHandler):
19 19     def log_message(self, *_: object) -> None:
20 20         pass
21 21
22 22     def _json(self, code: int, obj: dict) -> None:
23 23         body = json.dumps(obj).encode()
24 24         self.send_response(code)
25 25         self.send_header("Content-Type", "application/json")
26 26         self.send_header("Content-Length", str(len(body)))
27 27         self.end_headers()
28 28         self.wfile.write(body)
29 29
30 30     def _meta(self, q: dict) -> Meta:
31 31         """Build Meta from query params with filename fallback; raise on unknown."""
32 32         name = q.get("name", [""])[0]
33 33         m = parse_name(name)
34 34         meta = Meta(float(q["fs"][0]) if "fs" in q else m.fs,
35 35                    q.get("fmt", [m.fmt])[0], q.get("dtype", [m.dtype])[0],
36 36                    q.get("endian", [m.endian])[0],
37 37                    q.get("bitrev", ["1" if m.bitrev else "0"])[0] == "1",
38 38                    rf_center=float(q["rf"][0]) if "rf" in q else m.rf_center)
39 39         if not meta.ok():
40 40             raise ValueError(f"샘플레이트/포맷을 알 수 없습니다: {name}")
41 41         return meta
42 42
43 43     def do_POST(self) -> None: # noqa: N802 (BaseHTTPRequestHandler API)
44 44         url = urlparse(self.path)
45 45         if url.path not in ("/api/analyze", "/api/survey"):
46 46             self._json(404, {"error": "not found"})
47 47             return
48 48         q = parse_qs(url.query)
49 49         n = int(self.headers.get("Content-Length") or 0)
50 50         if n <= 0:
51 51             self._json(411, {"error": "Content-Length 필요"})
52 52             return
53 53         if n > _MAX_BODY:
54 54             self._json(413, {"error": "파일이 너무 큼니다 (최대 256MB)"})
55 55             return
56 56         try:
57 57             meta = self._meta(q)
58 58             burst = int(q["burst"][0]) if "burst" in q else None
59 59             x = decode(self.rfile.read(n), meta)

```

```

60 60         if x.size < 256:
61 61             raise ValueError("신호가 너무 짧습니다")
62 62     except ValueError as exc:
63 63         self._json(400, {"error": str(exc)})
64 64         return
65 65     try:
66 66         out = survey_web(x, meta) if url.path == "/api/survey" \
67 67             else analyze(x, meta, burst=burst).to_json()
68 68         self._json(200, out)
69 69     except Exception as exc: # surface DSP failures to the UI
70 70         self._json(500, {"error": f"분석 실패: {exc}"})
71 71
72 72     def do_GET(self) -> None: # noqa: N802
73 73         rel = urlparse(self.path).path.lstrip("/") or "index.html"
74 74         target = (_WEB / rel).resolve()
75 75         if not (target.is_file() and target.is_relative_to(_WEB.resolve())):
76 76             self._json(404, {"error": "not found"})
77 77             return
78 78         body = target.read_bytes()
79 79         self.send_response(200)
80 80         self.send_header("Content-Type", _MIME.get(target.suffix, "application/octet-stream"))
81 81         self.send_header("Content-Length", str(len(body)))
82 82         self.end_headers()
83 83         self.wfile.write(body)
84 84
85 85
86 86     def run(host: str = "127.0.0.1", port: int = 8000) -> None:
87 87         srv = ThreadingHTTPServer((host, port), Handler)
88 88         print(f"signus UI → http://{host}:{srv.server_address[1]}")
89 89         srv.serve_forever()

```

signus/web/index.html (신규 파일) +174 · 코드북 57쪽

```

1 + <!doctype html>
2 + <html lang="ko">
3 + <head>
4 + <meta charset="utf-8">
5 + <meta name="viewport" content="width=device-width, initial-scale=1">
6 + <title>Signus - 신호 복조 분석기</title>
7 + <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='
4 + 0 0 16 16'%3E%3Crect width='16' height='16' rx='5' fill='%233182F6'/%3E%3C/svg%3E">
8 + <link rel="stylesheet" href="/style.css">
9 + <script defer src="/app.js"></script>
10 + </head>
11 + <body>
12 + <header class="topbar">
13 + <div class="brand">
14 + <span class="logo" aria-hidden="true"></span>
15 + <div>
16 + <h1>Signus</h1>
17 + <p class="tagline">블라인드 PSK/QAM/FSK 신호 복조 분석기</p>
18 + </div>
19 + </div>
20 + </header>
21 +
22 + <main class="wrap">
23 + <section id="dropCard" class="card drop" tabindex="0" role="button"
24 + <div class="drop-icon" aria-hidden="true"></div>
25 + <input id="fileInput" type="file" multiple hidden>
26 + <p class="drop-title">신호 파일을 여기에 놓으세요</p>
27 +

```

```

28 +     <p class="drop-sub">여러 개를 놓으면 일괄 분석 · <code>.json</code> 정답 · <code>.sigmf-meta
   +     </code> 지원</p>
29 + </section>
30 +
31 + <section id="metaForm" class="card form hidden">
32 +     <p class="form-title">파일 이름에서 정보를 찾지 못했어요. 직접 입력해주세요.</p>
33 +     <div class="form-row">
34 +         <label>샘플레이트 (Hz)
35 +         <input id="mFs" type="number" min="1" step="any" placeholder="예: 1000000"></label>
36 +         <label>포맷
37 +         <select id="mFmt">
38 +             <option value="">선택</option><option value="iq">IQ</option><option value="real">Real<
   +             /option>
39 +         </select></label>
40 +         <label>데이터 타입
41 +         <select id="mDtype">
42 +             <option value="i16">i16 (16t)</option><option value="i8">i8 (8t)</option>
43 +             <option value="u8">u8 (8o)</option><option value="u16">u16 (16o)</option>
44 +             <option value="f32">f32</option><option value="f64">f64</option>
45 +         </select></label>
46 +     </div>
47 +     <button id="metaGo" class="btn">분석 시작</button>
48 + </section>
49 +
50 + <section id="loading" class="card loading hidden" aria-live="polite">
51 +     <div class="spinner" aria-hidden="true"></div>
52 +     <p id="loadMsg">신호를 분석하고 있어요...</p>
53 + </section>
54 +
55 + <section id="errorCard" class="card error hidden" role="alert">
56 +     <div class="err-mark" aria-hidden="true">!</div>
57 +     <div>
58 +         <p class="err-title">분석에 실패했어요</p>
59 +         <p id="errMsg" class="err-msg"></p>
60 +     </div>
61 + </section>
62 +
63 + <!-- Batch results table -->
64 + <section id="batchCard" class="card hidden">
65 +     <div class="card-head">
66 +         <h2 class="card-title">일괄 분석 결과</h2>
67 +         <button id="dlCsv" class="btn ghost small">CSV 내보내기</button>
68 +     </div>
69 +     <div class="table-wrap">
70 +         <table class="batch">
71 +             <thead><tr><th>파일</th><th>변조</th><th>중심주파수</th><th>심볼레이트</th><th>Lock</th>
   +             </tr></thead>
72 +             <tbody id="batchBody"></tbody>
73 +         </table>
74 +     </div>
75 +     <p class="hint">행을 클릭하면 상세 화면으로 이동합니다.</p>
76 + </section>
77 +
78 + <!-- Wideband survey: overview waterfall with detected-emitter boxes -->
79 + <section id="surveyCard" class="card hidden">
80 +     <div class="card-head">
81 +         <h2 class="card-title">광대역 Survey</h2>
82 +         <span id="survCount" class="pill ok"></span>
83 +     </div>
84 +     <div class="wf-wrap">

```

```

85 + <canvas id="survFall" class="fall"></canvas>
86 + <div id="survBoxes" class="boxes"></div>
87 + </div>
88 + <p class="hint">상자 또는 아래 목록을 클릭하면 그 신호의 상세로 이동합니다. (세로=시간, 가로
  4 +   =주파수)</p>
89 + <div id="survInfo" class="survinfo hidden"></div>
90 + <div class="table-wrap">
91 +   <table class="batch">
92 +     <thead><tr><th>중심주파수</th><th>종류</th><th>변조</th><th>심볼레이트</th><th>Lock</th>
  4 +       </tr></thead>
93 +     <tbody id="survBody"></tbody>
94 +   </table>
95 + </div>
96 + </section>
97 +
98 + <section id="results" class="results hidden">
99 +   <button id="backBatch" class="btn ghost small hidden">← 목록으로</button>
100 +
101 +   <div class="card summary">
102 +     <div class="summary-head">
103 +       <span id="statusPill" class="pill"></span>
104 +       <p id="fileName" class="file-name"></p>
105 +     </div>
106 +     <div class="summary-body">
107 +       <div class="gauge">
108 +         <svg viewBox="0 0 120 120" class="donut" aria-hidden="true">
109 +           <circle class="donut-track" cx="60" cy="60" r="52"></circle>
110 +           <circle id="donutArc" class="donut-arc" cx="60" cy="60" r="52"></circle>
111 +         </svg>
112 +         <div class="gauge-center"><span id="lockNum" class="gauge-num">0</span>
113 +           <span class="gauge-cap tip" title="별자리가 얼마나 조밀·정렬됐는지 (0~100). 60 이상
  4 +             이면 복조 신뢰">LOCK</span></div>
114 +         </div>
115 +         <div class="metrics">
116 +           <div class="metric"><span class="metric-label tip" title="변조 오차비 - 높을수록 깨
  4 +             끓 (dB)">MER</span>
117 +             <span id="merVal" class="metric-val"></span></div>
118 +           <div class="metric"><span class="metric-label tip" title="오차 벡터 크기 - 낮을수록 좋
  4 +             음 (%)>EVM</span>
119 +             <span id="evmVal" class="metric-val"></span></div>
120 +           <div class="metric"><span class="metric-label tip" title="추정 심볼 신호 대 잡음비 (dB
  4 +             )">SNR</span>
121 +             <span id="snrVal" class="metric-val"></span></div>
122 +         </div>
123 +       </div>
124 +       <div id="flagRow" class="chips"></div>
125 +       <div id="burstRow" class="chips"></div>
126 +     </div>
127 +
128 +     <div id="burstMap" class="card hidden">
129 +       <h2 class="card-title">버스트 지도</h2>
130 +       <div class="wf-wrap">
131 +         <canvas id="burstFall" class="fall"></canvas>
132 +         <div id="burstBoxes" class="boxes"></div>
133 +       </div>
134 +       <p class="hint">전체 녹음의 시간-주파수. 버스트 구간을 클릭하면 그 구간을 분석합니다.</p>
135 +     </div>
136 +
137 +     <div class="card">
138 +       <h2 class="card-title">스펙트럼 · 워터폴</h2>

```

```

139 + <canvas id="specCanvas" class="spec"></canvas>
140 + <canvas id="fallCanvas" class="fall"></canvas>
141 + </div>
142 +
143 + <div class="card const-card">
144 +   <div class="card-head">
145 +     <h2 class="card-title">성상도</h2>
146 +     <div class="play-row">
147 +       <button id="playBtn" class="btn ghost small" aria-label="재생/일시정지">⏮ 일시정지</bu
148 +         tton>
149 +       <select id="speedSel" class="sel small" aria-label="재생 속도">
150 +         <option value="1">1x</option><option value="4" selected>4x</option>
151 +         <option value="16">16x</option>
152 +       </select>
153 +     </div>
154 +   </div>
155 +   <canvas id="constCanvas" class="const"></canvas>
156 +   <input id="scrub" class="scrub" type="range" min="0" max="1000" value="0"
157 +     aria-label="재생 위치">
158 +   <p id="playInfo" class="hint"></p>
159 + </div>
160 +
161 + <div id="paramGrid" class="param-grid"></div>
162 +
163 + <div class="card downloads">
164 +   <h2 class="card-title">다운로드</h2>
165 +   <div class="dl-row">
166 +     <button id="copyBits" class="btn ghost">비트 복사</button>
167 +     <button id="dlBits" class="btn ghost">비트 .txt</button>
168 +     <button id="dlSyms" class="btn ghost">심볼 .csv</button>
169 +     <button id="dlReport" class="btn ghost">리포트 .json</button>
170 +   </div>
171 + </div>
172 + </section>
173 + </main>
174 + </body>
175 + </html>

```

signus/web/style.css (신규 파일) +243 · 코드북 61쪽

```

1 + :root {
2 +   --bg: #f4f6f8; --card: #fff; --ink: #191f28; --sub: #6b7684;
3 +   --blue: #3182F6; --blue-weak: #e8f1fe; --line: #eef1f4;
4 +   --green: #12b886; --amber: #f59f00; --red: #fa5252;
5 +   --field: #fff; --hover: #f7fafd; --border: #dfe4ea; /* theme-sensitive surfaces */
6 +   --ok-bg: #e6f7f1; --warn-bg: #fff4e0; --bad-bg: #ffecec;
7 +   --radius: 20px; --shadow: 0 6px 24px rgba(30, 42, 66, .07);
8 + }
9 + /* dark mode: follows the OS. The signal canvases are already dark, so they blend in. */
10 + @media (prefers-color-scheme: dark) {
11 +   :root {
12 +     --bg: #0e131b; --card: #161d28; --ink: #e6ebf2; --sub: #94a1b2;
13 +     --blue-weak: #17273c; --line: #232c39; --field: #1b2330; --hover: #1d2632;
14 +     --border: #2b3543; --ok-bg: #102c24; --warn-bg: #302512; --bad-bg: #321b1e;
15 +     --shadow: 0 6px 24px rgba(0, 0, 0, .38);
16 +   }
17 + }
18 +
19 + * { box-sizing: border-box; }
20 +
21 + body {

```

```

22 + margin: 0; background: var(--bg); color: var(--ink);
23 + font-family: -apple-system, 'Apple SD Gothic Neo', 'Segoe UI', 'Malgun Gothic', sans-serif;
24 + line-height: 1.5; -webkit-font-smoothing: antialiased;
25 + }
26 +
27 + .wrap { max-width: 580px; margin: 0 auto; padding: 8px 20px 64px; }
28 +
29 + .topbar { max-width: 580px; margin: 0 auto; padding: 28px 20px 12px; }
30 + .brand { display: flex; align-items: center; gap: 12px; }
31 + .logo {
32 +   width: 40px; height: 40px; border-radius: 12px; flex: none;
33 +   background: linear-gradient(135deg, var(--blue), #6aa8ff);
34 + }
35 + h1 { margin: 0; font-size: 22px; font-weight: 800; letter-spacing: -.4px; }
36 + .tagline { margin: 2px 0 0; color: var(--sub); font-size: 13px; }
37 +
38 + .card {
39 +   background: var(--card); border-radius: var(--radius); padding: 20px;
40 +   box-shadow: var(--shadow); margin-top: 16px; animation: rise .35s ease both;
41 + }
42 + .card-title { margin: 0 0 12px; font-size: 15px; font-weight: 700; }
43 + .hidden { display: none !important; }
44 +
45 + @keyframes rise { from { opacity: 0; transform: translateY(10px); } to { opacity: 1; transform
46 +   : none; } }
47 +
48 + /* Dropzone */
49 + .drop {
50 +   text-align: center; cursor: pointer; border: 2px dashed var(--border);
51 +   transition: border-color .18s, background .18s, transform .18s;
52 + }
53 + .drop:hover, .drop:focus-visible { border-color: var(--blue); background: var(--hover); outline
54 +   : none; }
55 + .drop.drag { border-color: var(--blue); background: var(--blue-weak); transform: scale(1.01); }
56 + .drop-icon {
57 +   width: 52px; height: 52px; margin: 6px auto 14px; border-radius: 16px;
58 +   background: var(--blue-weak); position: relative;
59 + }
60 + .drop-icon::after {
61 +   content: ''; position: absolute; inset: 16px 15px; border: 3px solid var(--blue);
62 +   border-radius: 3px; border-top: none; border-right: none;
63 +   transform: rotate(45deg) translate(2px, -2px);
64 + }
65 + .drop-title { margin: 0; font-size: 16px; font-weight: 700; }
66 + .drop-sub { margin: 6px 0 0; color: var(--sub); font-size: 13px; }
67 + code { background: var(--line); padding: 1px 6px; border-radius: 6px; font-size: 12px; }
68 +
69 + /* Meta form */
70 + .form-title { margin: 0 0 12px; font-size: 14px; font-weight: 600; }
71 + .form-row { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; }
72 + .form-row label { display: flex; flex-direction: column; gap: 6px; font-size: 12px; color: var(
73 +   --sub); }
74 + input, select {
75 +   font: inherit; padding: 10px 12px; border: 1px solid var(--border); border-radius: 12px;
76 +   background: var(--field); color: var(--ink);
77 + }
78 + input:focus, select:focus { outline: 2px solid var(--blue); border-color: transparent; }
79 +
80 + .btn {
81 +   margin-top: 16px; width: 100%; padding: 13px; border: none; border-radius: 14px;

```

```

79 + background: var(--blue); color: #fff; font-size: 15px; font-weight: 700; cursor: pointer;
80 + transition: filter .15s, transform .1s;
81 + }
82 + .btn:hover { filter: brightness(1.05); }
83 + .btn:active { transform: scale(.98); }
84 + .btn.ghost { background: var(--blue-weak); color: var(--blue); margin-top: 0; }
85 +
86 + /* Loading */
87 + .loading { display: flex; align-items: center; gap: 14px; color: var(--sub); }
88 + .spinner {
89 +   width: 26px; height: 26px; border: 3px solid var(--line);
90 +   border-top-color: var(--blue); border-radius: 50%; animation: spin .8s linear infinite;
91 + }
92 + @keyframes spin { to { transform: rotate(360deg); } }
93 +
94 + /* Error */
95 + .error { display: flex; gap: 14px; align-items: center; border-left: 4px solid var(--red); }
96 + .err-mark {
97 +   width: 34px; height: 34px; flex: none; border-radius: 50%; background: var(--bad-bg);
98 +   color: var(--red); font-weight: 800; display: grid; place-items: center;
99 + }
100 + .err-title { margin: 0; font-weight: 700; }
101 + .err-msg { margin: 2px 0 0; color: var(--sub); font-size: 14px; }
102 +
103 + /* Summary + gauge */
104 + .summary-head { display: flex; align-items: center; gap: 12px; margin-bottom: 16px; }
105 + .pill { padding: 6px 14px; border-radius: 999px; font-size: 13px; font-weight: 700; }
106 + .pill.ok { background: var(--ok-bg); color: var(--green); }
107 + .pill.warn { background: var(--warn-bg); color: var(--amber); }
108 + .pill.bad { background: var(--bad-bg); color: var(--red); }
109 + .file-name { margin: 0; font-size: 13px; color: var(--sub); overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
110 +
111 + .summary-body { display: flex; align-items: center; gap: 24px; }
112 + .gauge { position: relative; width: 120px; height: 120px; flex: none; }
113 + .donut { width: 120px; height: 120px; }
114 + .donut-track { fill: none; stroke: var(--line); stroke-width: 12; }
115 + .donut-arc {
116 +   fill: none; stroke: var(--blue); stroke-width: 12; stroke-linecap: round;
117 +   transform: rotate(-90deg); transform-origin: center;
118 +   transition: stroke-dashoffset .9s cubic-bezier(.22, .61, .36, 1);
119 + }
120 + .gauge-center { position: absolute; inset: 0; display: grid; place-items: center; }
121 + .gauge-num { font-size: 34px; font-weight: 800; line-height: 1; font-variant-numeric: tabular-nums; }
122 + .gauge-cap { font-size: 11px; color: var(--sub); letter-spacing: 1px; }
123 +
124 + .metrics { display: flex; flex-direction: column; gap: 12px; }
125 + .metric { display: flex; flex-direction: column; gap: 2px; }
126 + .metric-label { font-size: 12px; color: var(--sub); }
127 + .tip { cursor: help; text-decoration: underline dotted; text-underline-offset: 3px;
128 +   text-decoration-color: color-mix(in srgb, var(--sub) 45%, transparent); }
129 + .metric-val { font-size: 20px; font-weight: 700; font-variant-numeric: tabular-nums; }
130 +
131 + /* Card head with actions: spacing lives on the CONTAINER so the title and the
132 +   action stay vertically centered on the same line */
133 + .card-head { display: flex; align-items: center; justify-content: space-between;
134 +   gap: 12px; margin-bottom: 12px; }
135 + .card-head .card-title { margin-bottom: 0; }
136 + .btn.small { width: auto; margin-top: 0; padding: 8px 14px; font-size: 13px; border-radius: 10px

```

```

137 + ; }
138 + .sel.small {
139 +   font: inherit; font-size: 13px; font-weight: 600; padding: 7px 10px; border-radius: 10px;
140 +   border: 1px solid var(--border); background: var(--field); color: var(--ink);
141 + }
142 + .play-row { display: flex; gap: 8px; align-items: center; }
143 + .hint { margin: 8px 0 0; font-size: 12px; color: var(--sub); }
144 + /* Spectrum + waterfall */
145 + .spec { width: 100%; height: 120px; display: block; border-radius: 12px 12px 0 0; background: #0
146 +   d1320; }
147 + .fall { width: 100%; height: 150px; display: block; border-radius: 0 0 12px 12px; background: #0
148 +   d1320; }
149 + /* Constellation + playback */
150 + .const { width: 100%; aspect-ratio: 1; border-radius: 14px; display: block; background: #0d1320
151 +   ; }
152 + .scrub { width: 100%; margin-top: 12px; accent-color: var(--blue); }
153 + /* Batch table */
154 + .table-wrap { overflow-x: auto; }
155 + table.batch { width: 100%; border-collapse: collapse; font-size: 13.5px; }
156 + table.batch th {
157 +   text-align: left; font-size: 12px; color: var(--sub); font-weight: 600;
158 +   padding: 8px 10px; border-bottom: 1px solid var(--line);
159 + }
160 + table.batch td { padding: 11px 10px; border-bottom: 1px solid var(--line);
161 +   font-variant-numeric: tabular-nums; }
162 + table.batch .pill { padding: 3px 10px; font-size: 12px; }
163 + table.batch tbody tr { cursor: pointer; transition: background .12s; }
164 + table.batch tbody tr:hover { background: var(--hover); }
165 + table.batch .fn { font-weight: 600; max-width: 210px; overflow: hidden;
166 +   text-overflow: ellipsis; white-space: nowrap; }
167 + /* Param grid */
168 + .param-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 16px; }
169 + .param { padding: 16px; margin: 0; animation: rise .4s ease both; }
170 + .param-label { font-size: 12px; color: var(--sub); }
171 + .param-val { font-size: 22px; font-weight: 800; margin-top: 4px; letter-spacing: -.3px;
172 +   font-variant-numeric: tabular-nums; }
173 + .param-note { font-size: 11px; color: var(--sub); margin-top: 2px; }
174 + .chips { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 10px; }
175 + .chips:empty { display: none; } /* no flags -> no ghost margin */
176 + .chip { font-size: 11px; font-weight: 700; padding: 4px 9px; border-radius: 999px; }
177 + .chip.hit { background: var(--ok-bg); color: var(--green); }
178 + .chip.miss { background: var(--bad-bg); color: var(--red); }
179 + .chip.warn { background: var(--warn-bg); color: var(--amber); }
180 +
181 + /* Burst selector (chips that act) */
182 + .chip-btn {
183 +   font: inherit; font-size: 11px; font-weight: 700; padding: 4px 10px;
184 +   border-radius: 999px; border: 1px solid var(--border); background: var(--field);
185 +   color: var(--sub); cursor: pointer; transition: color .12s, border-color .12s;
186 + }
187 + .chip-btn:hover { border-color: var(--blue); color: var(--blue); }
188 + .chip-btn.on { background: var(--blue-weak); color: var(--blue); border-color: transparent; }
189 +
190 + /* Downloads */
191 + .dl-row { display: flex; gap: 10px; flex-wrap: wrap; }
192 + .dl-row .btn { flex: 1; min-width: 120px; }

```

```

193 +
194 + /* Survey overview: waterfall + detected-emitter boxes */
195 + .wf-wrap { position: relative; }
196 + .wf-wrap .fall { height: 230px; border-radius: 12px; }
197 + .boxes { position: absolute; inset: 0; pointer-events: none; }
198 + .box {
199 +   position: absolute; box-sizing: border-box; border: 2px solid var(--blue);
200 +   border-radius: 5px; background: rgba(49, 130, 246, .10); cursor: pointer;
201 +   pointer-events: auto; min-width: 7px; min-height: 12px;
202 +   transition: background .12s, box-shadow .12s;
203 + }
204 + .box:hover, .box:focus-visible {
205 +   background: rgba(49, 130, 246, .24); box-shadow: 0 0 0 2px rgba(255, 255, 255, .55); outline
206 +     : none;
207 + }
208 + .box-lbl { /* hidden by default (a spectrogram full of labels is unreadable); shown on hover */
209 +   position: absolute; top: -17px; left: -2px; white-space: nowrap; font-size: 10px;
210 +   font-weight: 700; color: #fff; background: rgba(13, 19, 32, .85); padding: 1px 6px;
211 +   border-radius: 6px; pointer-events: none; display: none; z-index: 2;
212 + }
213 + .box:hover .box-lbl, .box:focus-visible .box-lbl, .box.sel .box-lbl { display: block; }
214 + .box.sel { box-shadow: 0 0 0 2px #fff, 0 0 0 4px var(--blue); }
215 + .box.ok { border-color: var(--green); background: rgba(18, 184, 134, .13); }
216 + .box.ok .box-lbl { background: var(--green); }
217 + .box.warn { border-color: var(--amber); background: rgba(245, 159, 0, .13); }
218 + .box.warn .box-lbl { background: var(--amber); }
219 + .box.bad { border-color: var(--red); background: rgba(250, 82, 82, .13); }
220 + .box.bad .box-lbl { background: var(--red); }
221 + .box.gray { border-color: #8592a6; background: rgba(133, 146, 166, .15); }
222 + .box.gray .box-lbl { background: #8592a6; }
223 + .pill.gray { background: var(--line); color: var(--sub); }
224 + table.batch .fc { font-weight: 600; font-variant-numeric: tabular-nums; }
225 +
226 + /* Non-digital emitter inline info */
227 + .survinfo {
228 +   background: var(--hover); border-radius: 14px; padding: 14px 16px; margin: 12px 0;
229 +   border-left: 4px solid #8592a6; animation: rise .3s ease both;
230 + }
231 + .survinfo h3 { margin: 0 0 10px; font-size: 14px; }
232 + .si-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px 16px; }
233 + .si-k { font-size: 11px; color: var(--sub); }
234 + .si-v { font-size: 17px; font-weight: 700; font-variant-numeric: tabular-nums; }
235 +
236 + @media (max-width: 420px) {
237 +   .form-row { grid-template-columns: 1fr; }
238 +   .param-grid { grid-template-columns: 1fr; }
239 +   .summary-body { gap: 18px; }
240 + }
241 +
242 + @media (prefers-reduced-motion: reduce) {
243 +   * { animation: none !important; transition: none !important; }
244 + }

```

signus/web/app.js (신규 파일) +684 · 코드북 66쪽

```

1 + "use strict";
2 + const $ = (id) => document.getElementById(id);
3 + const API = "/api/analyze";
4 + const SURVEY_API = "/api/survey";
5 + const FS_RE = /(?:^|_)fs(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:_|$)/i;

```

```

6 + const RF_RE = /(?:^|_)rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:_|$)/i;
7 + const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
8 + const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
9 +               burst: null, survey: null, drilled: false, overview: null };
10 +
11 + /* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
12 + const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
13 + // baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
14 + const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "
15 +   f64" };
16 + function parseName(name) {
17 +   const stem = name.replace(/^.*/, "").replace(/\.^[^.]*/, "").toLowerCase();
18 +   const mt = FS_RE.exec(stem), rt = RF_RE.exec(stem), toks = stem.split("_");
19 +   const dt = toks.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
20 +   return {
21 +     fs: mt ? parseFloat(mt[1]) : null,
22 +     fmt: toks.find((t) => t === "iq" || t === "real") || null,
23 +     dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
24 +     endian: toks.includes("be") ? "be" : "le",
25 +     bitrev: toks.includes("bitrev"),
26 +     rf: rt ? parseFloat(rt[1]) : null,
27 +   };
28 + }
29 + /* SigMF: core:datatype -> our fmt/dtype/endian */
30 + const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
31 +               cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
32 + function metaFromSigmf(doc) {
33 +   const g = doc && doc.global;
34 +   if (!g || !g["core:datatype"]) return null;
35 +   const dt = g["core:datatype"], base = dt.replace(/_[\lb]e$/, "");
36 +   if (!(base in SIGMF)) return null;
37 +   const [fmt, dtype] = SIGMF[base];
38 +   const cap = (doc.captures || []).length ? {} : {};
39 +   return { fs: Number(g["core:sample_rate"]), fmt, dtype,
40 +           endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
41 +           rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
42 + }
43 + /* --- number formatting --- */
44 + function sig(x) {
45 +   const a = Math.abs(x);
46 +   return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
47 + }
48 + function fmtHz(v) {
49 +   if (v == null) return "-";
50 +   const a = Math.abs(v);
51 +   if (a >= 1e6) return sig(v / 1e6) + " MHz";
52 +   if (a >= 1e3) return sig(v / 1e3) + " kHz";
53 +   return sig(v) + " Hz";
54 + }
55 + function fmtRf(v) { // absolute RF: keep kHz resolution even in the 100s-of-MHz range
56 +   return (v / 1e6).toFixed(3) + " MHz";
57 + }
58 +
59 + /* --- ideal constellation points (mirror of constellations.ideal_points) --- */
60 + function idealPoints(mod) {
61 +   if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
62 +   if (mod === "qpsk" || mod === "8psk") {
63 +     const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
64 +     for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(of

```

```

4 +      f + 2 * Math.PI * k / m) });
65 +      return p;
66 +  }
67 +  const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
68 +  for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
69 +  for (const qi of lv) for (const ii of lv) {
70 +      if (mod === "32qam" && Math.abs(ii * qi) === 25) continue; // cross: corners removed
71 +      p.push({ i: ii, q: qi });
72 +  }
73 +  let e = 0;
74 +  for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
75 +  const norm = Math.sqrt(e / p.length);
76 +  return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
77 + }
78 +
79 + /* --- file intake: one data file (+sidecar) or many (batch) --- */
80 + async function acceptFiles(list) {
81 +     const files = [...list];
82 +     const metas = files.filter((f) => /\. (json|sigmf-meta)$/i.test(f.name));
83 +     const data = files.filter((f) => !metas.includes(f));
84 +     if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
85 +     if (data.length > 1) return runBatch(data, metas);
86 +
87 +     state.file = data[0];
88 +     state.resp = null;
89 +     state.batch = [];
90 +     state.burst = null;
91 +     state.survey = null;
92 +     state.drilled = false;
93 +     state.overview = null;
94 +     const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta"));
95 +     const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json"));
96 +     state.sidecar = truth ? await readJson(truth) : null; // client-side only, never sent
97 +     state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
98 +     if (state.meta && state.meta.fs && state.meta.fmt) runSurvey();
99 +     else showMetaForm();
100 + }
101 + const readJson = (f) => f.text().then(JSON.parse).catch(() => null);
102 +
103 + function showMetaForm() {
104 +     hideAll();
105 +     state.meta = state.meta || {};
106 +     if (state.meta.fs) $("mFs").value = state.meta.fs;
107 +     $("mFmt").value = state.meta.fmt || "";
108 +     $("mDtype").value = state.meta.dtype || "i16";
109 +     show($("#metaForm"));
110 + }
111 + $("metaGo").onclick = () => {
112 +     const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
113 +     if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
114 +     state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
115 +     runSurvey();
116 + };
117 +
118 + /* --- server call (contract-frozen) --- */
119 + function query(file, m, burst) {
120 +     const q = new URLSearchParams({ name: file.name });
121 +     if (m.fs) q.set("fs", m.fs);
122 +     if (m.fmt) q.set("fmt", m.fmt);
123 +     if (m.dtype) q.set("dtype", m.dtype);

```

```

124 +   if (m.endian) q.set("endian", m.endian);
125 +   if (m.bitrev) q.set("bitrev", "1");
126 +   if (m.rf != null) q.set("rf", m.rf);
127 +   if (burst != null) q.set("burst", burst);
128 +   return q;
129 + }
130 + async function postTo(api, file, m, burst) {
131 +   const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
132 +   const data = await res.json();
133 +   if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
134 +   return data;
135 + }
136 + async function runSurvey() {                                     // entry: one capture -> single detail OR survey
137 +   hideAll();
138 +   state.drilled = false;
139 +   $("loadMsg").textContent = "신호를 조사하고 있어요...";
140 +   show($("loading"));
141 +   try {
142 +     const resp = await postTo(SURVEY_API, state.file, state.meta);
143 +     if (resp.mode === "single") {
144 +       state.survey = null; state.overview = resp.overview || null; state.resp = resp.result;
145 +       render(resp.result);
146 +       if (state.overview) renderBurstMap(state.overview, resp.result);
147 +     } else renderSurvey(resp);
148 +   } catch (e) {
149 +     showError(e.message);
150 +   }
151 + }
152 + async function analyze() {                                       // burst re-selection within a single signal
153 +   hideAll();
154 +   $("loadMsg").textContent = "신호를 분석하고 있어요...";
155 +   show($("loading"));
156 +   try {
157 +     state.resp = await postTo(API, state.file, state.meta, state.burst);
158 +     render(state.resp);
159 +     if (state.overview) renderBurstMap(state.overview, state.resp);
160 +     $("backBatch").classList.toggle("hidden", !state.batch.length);
161 +   } catch (e) {
162 +     showError(e.message);
163 +   }
164 + }
165 +
166 + /* --- batch mode --- */
167 + async function runBatch(data, metas) {
168 +   hideAll();
169 +   show($("loading"));
170 +   const truths = new Map(metas.filter((m) => /\s.json$/i.test(m.name))
171 +     .map((m) => [m.name.replace(/\s.json$/i, ""), m]));
172 +   const sigmfs = new Map(metas.filter((m) => /\s.sigmf-meta$/i.test(m.name))
173 +     .map((m) => [m.name.replace(/\s.sigmf-meta$/i, ""), m]));
174 +   const rows = [];
175 +   for (let i = 0; i < data.length; i++) {
176 +     const f = data[i];
177 +     $("loadMsg").textContent = `일괄 분석 중... (${i + 1}/${data.length}) ${f.name}`;
178 +     // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
179 +     // beats filename tokens
180 +     const sm = sigmfs.get(f.name.replace(/\.[^.]*/i, ""));
181 +     const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
182 +     let resp = null, err = null;
183 +     try {

```

```

184 +     if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
185 +     resp = await postTo(API, f, meta);
186 +   } catch (e) { err = e.message; }
187 +   const tf = truths.get(f.name);
188 +   // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
189 +   // it used to read state.meta, which a fresh batch never set (crash) and a previous
190 +   // single-file run left stale (silent wrong-fs decode)
191 +   rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
192 + }
193 + state.batch = rows;
194 + hideAll();
195 + renderBatch(rows);
196 + }
197 +
198 + function renderBatch(rows) {
199 +   $("batchBody").innerHTML = rows.map((r, i) => {
200 +     if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
201 +       `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
202 +     const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
203 +     const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
204 +     return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
205 +       `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
206 +       `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
207 +   }).join("");
208 +   $("batchBody").querySelectorAll("tr").forEach((tr) => {
209 +     tr.onclick = () => {
210 +       const r = state.batch[+tr.dataset.i];
211 +       if (!r.resp) return;
212 +       state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
213 +       state.burst = null; state.drilled = false; state.survey = null; state.overview = null;
214 +       hideAll();
215 +       render(r.resp);
216 +       const b = $("backBatch");
217 +       b.textContent = "← 목록으로";
218 +       b.onclick = () => { hideAll(); renderBatch(state.batch); };
219 +       b.classList.remove("hidden");
220 +     };
221 +   });
222 +   show($("batchCard"));
223 + }
224 + $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
225 + $("dlCsv").onclick = () => {
226 +   const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
227 +   const body = state.batch.filter((r) => r.resp).map((r) => {
228 +     const d = r.resp.detected, q = r.resp.quality;
229 +     return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
230 +       r.resp.snr_est_db ?? ""].join(",");
231 +   }).join("\n");
232 +   saveBlob("signus_batch.csv", head + body, "text/csv");
233 + };
234 +
235 + /* --- wideband survey overview (waterfall + detected-emitter boxes) --- */
236 + function renderSurvey(resp) {
237 +   hideAll();
238 +   stopPlay();
239 +   state.survey = resp;
240 +   const dig = resp.emitters.filter((e) => e.result).length;
241 +   $("survCount").textContent = `${resp.emitters.length}개 신호 · 디지털 ${dig}`;
242 +   $("survInfo").classList.add("hidden");
243 +   renderSurveyList(resp);

```

```

244 + show($("#surveyCard")); // show first so the canvas has a real width
245 + paintWaterfall($("#survFall"), resp.overview.waterfall, 230);
246 + drawBoxes(resp);
247 + }
248 +
249 + function boxStyle(e) { // -> [css class, short label]
250 +   if (e.result) {
251 +     const lock = Math.round(e.result.quality.lock);
252 +     return [lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad",
253 +       `${e.result.detected.mod.toUpperCase()} · ${lock}`];
254 +   }
255 +   if (e.kind === "chirp") {
256 +     return ["gray", e.info && e.info.sf ? `LoRa SF${e.info.sf}` : "처프"];
257 +   }
258 +   return ["gray", { analog: "아날로그", tone: "톤/CW", tooshort: "짧음",
259 +     error: "오류" }[e.kind] || e.kind];
260 + }
261 +
262 + function drawBoxes(resp) {
263 +   const host = $("#survBoxes"), fs = resp.overview.fs, n = resp.overview.n || 1;
264 +   host.innerHTML = "";
265 +   resp.emitters.forEach((e) => {
266 +     const d = e.det, [cls, lbl] = boxStyle(e);
267 +     const wpct = Math.max(0.8, d.bw / fs * 100); // width = bandwidth / fs
268 +     const left = (d.fc + fs / 2) / fs * 100 - wpct / 2; // x = carrier on -fs/2..fs/2
269 +     const top = Math.max(0, d.t0 / n * 100); // y = burst start over capture
270 +     const b = document.createElement("button");
271 +     b.className = "box " + cls;
272 +     b.style.left = Math.max(0, Math.min(100 - wpct, left)) + "%";
273 +     b.style.width = wpct + "%";
274 +     b.style.top = top + "%";
275 +     b.style.height = Math.min(100 - top, Math.max(7, (d.t1 - d.t0) / n * 100)) + "%";
276 +     b.title = lbl;
277 +     b.innerHTML = `<span class="box-lbl">${lbl}</span>`;
278 +     b.onclick = () => drillEmitter(e);
279 +     host.appendChild(b);
280 +   });
281 + }
282 +
283 + function renderSurveyList(resp) {
284 +   const KIND = { linear: "디지털", fsk: "FSK", chirp: "처프/LoRa", analog: "아날로그",
285 +     tone: "톤/CW", tooshort: "짧음", error: "오류" };
286 +   const rf0 = resp.rf_center; // real RF = capture centre + abs_fc
287 +   $("#survBody").innerHTML = resp.emitters.map((e, i) => {
288 +     const [cls] = boxStyle(e), r = e.result;
289 +     const lock = r ? `<span class="pill ${cls}">${Math.round(r.quality.lock)}</span>`
290 +       : `<span class="pill gray"></span>`;
291 +     return `<tr data-i="${i}"><td class="fc">${rf0 != null ? fmtRf(rf0 + e.abs_fc) : fmtHz(e.abs
292 +       _fc)}</td>` +
293 +       `<td>${KIND[e.kind] || e.kind}</td><td>${r ? r.detected.mod.toUpperCase() : "-"}</td>` +
294 +       `<td>${r ? fmtHz(r.detected.baud) : "-"}</td><td>${lock}</td></tr>`;
295 +   }).join("");
296 +   $("#survBody").querySelectorAll("tr").forEach((tr) => {
297 +     tr.onclick = () => drillEmitter(resp.emitters[+tr.dataset.i]);
298 +   });
299 + }
300 +
301 + function drillEmitter(e) {
302 +   if (e.result) { // digital -> reuse the single-signal detail view
303 +     state.drilled = true;

```

```

303 +     state.overview = null; // a cut channel has no whole-capture burst map
304 +     state.resp = e.result;
305 +     render(e.result);
306 +     const rf0 = state.survey && state.survey.rf_center;
307 +     $("fileName").textContent =
308 +         `에미터 @ ${rf0 != null ? fmtRf(rf0 + e.abs_fc) : fmtHz(e.abs_fc)} · ${state.file.name}`;
309 +     const b = $("backBatch");
310 +     b.textContent = "← Survey로";
311 +     b.onclick = backToSurvey;
312 +     b.classList.remove("hidden");
313 + } else { // non-digital -> inline info, stay on the overview
314 +     showSurveyInfo(e);
315 + }
316 + }
317 + function backToSurvey() { state.drilled = false; renderSurvey(state.survey); }
318 +
319 + function showSurveyInfo(e) {
320 +     const d = e.det;
321 +     let title, rows;
322 +     if (e.kind === "chirp" && e.info) { // linear chirp / CSS (LoRa): characterized only
323 +         const c = e.info;
324 +         title = (c.sf ? `LoRa 계열 (SF${c.sf})` : "선형 처프 / CSS") + " - 복조 대상 아님";
325 +         // rate + direction are robust (coherent from the beat tone); bw/SF only when the
326 +         // channel is cleanly isolated enough to snap to a LoRa hypothesis.
327 +         rows = [
328 +             ["중심주파수", fmtHz(e.abs_fc)],
329 +             ["처프 방향", c.up ? "▲ 상승" : "▼ 하강"],
330 +             ["처프율", (c.mu / 1e9).toFixed(3) + " MHz/ms"];
331 +             if (c.sf) rows.push(["대역폭", fmtHz(c.bw)], ["심볼레이트", fmtHz(c.rs)],
332 +                 ["심볼시간", (c.tsym * 1e3).toFixed(2) + " ms"]);
333 +         } else {
334 +             const KIND = { analog: "아날로그 (FM/음성)", tone: "톤 / CW",
335 +                 tooshort: "너무 짧은 버스트", error: "분석 오류" };
336 +             title = (KIND[e.kind] || e.kind) + " - 복조 대상 아님";
337 +             rows = [
338 +                 ["중심주파수", fmtHz(e.abs_fc)], ["대역폭", fmtHz(d.bw)],
339 +                 ["SNR", d.snr_db == null ? "-" : d.snr_db.toFixed(1) + " dB"],
340 +                 ["심볼레이트(추정)", fmtHz(d.baud_hint)];
341 +             }
342 +             $("survInfo").innerHTML = `<h3>${title}</h3>` +
343 +                 `<div class="si-grid">` + rows.map(([k, v]) =>
344 +                     `<div><div class="si-k">${k}</div><div class="si-v">${v}</div></div>`).join("") + "</div>"
345 +                 ;
346 +             $("survInfo").classList.remove("hidden");
347 +             $("survInfo").scrollIntoView({ behavior: REDUCED ? "auto" : "smooth", block: "nearest" });
348 +         }
349 +     }
350 +
351 +     /* --- burst map: whole-record waterfall with clickable time-burst boxes (single signal) --- */
352 +     function renderBurstMap(ov, result) {
353 +         show($("burstMap"));
354 +         paintWaterfall($("burstFall"), ov.waterfall, 150);
355 +         const host = $("burstBoxes"), n = ov.n || 1;
356 +         host.innerHTML = "";
357 +         (result.bursts || []).forEach((b, i) => {
358 +             const top = Math.max(0, b.start / n * 100);
359 +             const dur = (b.end - b.start) / ov.fs;
360 +             const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0)
361 +                 } + " ms"}`;
362 +             const box = document.createElement("button"); // full width (one signal), spans t0..t1 i
363 +                 n time
364 +             box.className = "box" + (i === result.burst_idx ? " sel" : "");
365 +             box.style.left = "0%"; box.style.width = "100%"; box.style.top = top + "%";

```

```

360 +     box.style.height = Math.min(100 - top, Math.max(5, (b.end - b.start) / n * 100)) + "%";
361 +     box.title = lbl;
362 +     box.innerHTML = `${lbl}</span>`;
363 +     box.onclick = () => { state.burst = i; analyze(); }; // re-analyse that burst; map persist
364 +     host.appendChild(box);
365 + });
366 + }
367 +
368 + /* --- rendering --- */
369 + function statusOf(lock) {
370 +     if (lock >= 60) return ["복조 성공", "ok"];
371 +     if (lock >= 40) return ["부분 복조", "warn"];
372 +     return ["복조 실패", "bad"];
373 + }
374 + function render(d) {
375 +     hideAll();
376 +     show($("#results"));
377 +     $("#burstMap").classList.add("hidden"); // re-shown by renderBurstMap only in multi-burst sin
378 +     // gle mode
379 +     const lock = d.quality.lock, [txt, cls] = statusOf(lock);
380 +     const pill = $("#statusPill");
381 +     pill.textContent = txt;
382 +     pill.className = "pill " + cls;
383 +     $("#fileName").textContent =
384 +         `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
385 +     $("#merVal").textContent = d.quality.mer_db == null ? "-" : d.quality.mer_db.toFixed(1) + " dB"
386 +     ;
387 +     $("#evmVal").textContent = d.quality.evm == null ? "-" : (d.quality.evm * 100).toFixed(1) + " %"
388 +     ;
389 +     $("#snrVal").textContent = snrText(d);
390 +
391 +     const flags = [];
392 +     if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
393 +     if (d.eq && d.eq.applied) flags.push('<span class="chip hit">등화기 적용 · ${
394 +         d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"></span>');
395 +     if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>');
396 +     if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span>');
397 +     if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어
398 +         싱 가능</span>');
399 +     $("#flagRow").innerHTML = flags.join("");
400 +
401 +     renderBursts(d);
402 +     animateGauge(lock, cls);
403 +     renderParams(d);
404 +     drawSpectrum(d);
405 +     drawWaterfall(d);
406 +     startPlay(d);
407 + }
408 + function snrText(d) {
409 +     const s = d.snr_est_db;
410 +     if (s == null || d.quality.lock < 30) return "-";
411 +     return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
412 + }
413 + function renderBursts(d) {
414 +     const row = $("#burstRow"), bs = d.bursts || [];
415 +     // a survey-drilled emitter is already a cut channel: burst re-selection would

```

```

413 + // re-post the whole capture, so it is disabled while drilled. When the burst MAP
414 + // is shown (multi-burst single signal) the map replaces the chips.
415 + if (state.drilled || state.overview || bs.length < 2) { row.innerHTML = ""; return; }
416 + const dur = (b) => {
417 +   const sec = (b.end - b.start) / d.fs;
418 +   return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
419 + };
420 + row.innerHTML = bs.map((b, i) =>
421 +   `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
422 +   `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
423 + row.querySelectorAll("button").forEach((el) => {
424 +   el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
425 + });
426 + }
427 +
428 + function chip(hit, truthTxt) {
429 +   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치"}</span>`;
430 + }
431 + function renderParams(d) {
432 +   const det = d.detected, t = state.sidecar && state.sidecar.truth;
433 +   const near = (a, b, tol) => Math.abs(a - b) <= tol;
434 +   const rows = [
435 +     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
436 +       t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
437 +       det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
438 +     ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.baud), fmtHz(t.baud)),
439 +       det.baud_conf ? `스펙트럴 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
440 +     ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUpperCase(), t.mod.toUpperCase()),
441 +       det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
442 +   ];
443 +   if (d.family === "fsk") {
444 +     rows.push(["변조지수 h", det.h.toFixed(2),
445 +       t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
446 +   } else {
447 +     rows.push(["롤오프", det.rolloff.toFixed(2),
448 +       t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2)), ""]);
449 +   }
450 +   $("paramGrid").innerHTML = rows.map(([label, val, hit, note]) => `
451 +     <div class="card param">
452 +       <div class="param-label">${label}</div>
453 +       <div class="param-val">${val}</div>
454 +       ${note ? `<div class="param-note">${note}</div>` : ""}
455 +       ${hit ? `<div class="chips">${hit}</div>` : ""}
456 +     </div>`).join("");
457 + }
458 +
459 + function animateGauge(lock, cls) {
460 +   const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
461 +   arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
462 +   arc.style.strokeDasharray = C;
463 +   const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
464 +   if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return; }
465 +   arc.style.strokeDashoffset = C;
466 +   requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
467 +   const t0 = performance.now();

```

```

468 +   const step = (t) => {
469 +     const k = Math.min(1, (t - t0) / 900);
470 +     num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
471 +     if (k < 1) requestAnimationFrame(step);
472 +   };
473 +   requestAnimationFrame(step);
474 + }
475 +
476 + /* --- canvas helpers --- */
477 + function fit(cv, hCss) {
478 +   const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
479 +   const h = hCss || w;
480 +   cv.width = w * dpr; cv.height = h * dpr;
481 +   const g = cv.getContext("2d");
482 +   g.setTransform(dpr, 0, 0, dpr, 0, 0);
483 +   return [g, w, h];
484 + }
485 + function pct(a, p) {
486 +   const s = [...a].sort((x, y) => x - y);
487 +   return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
488 + }
489 +
490 + /* --- spectrum --- */
491 + function drawSpectrum(d) {
492 +   const [g, w, h] = fit($("#specCanvas"), 120);
493 +   g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
494 +   if (!d.spectrum) return;
495 +   const f = d.spectrum.f, db = d.spectrum.db;
496 +   const lo = pct(db, 0.02), hi = pct(db, 1) + 2;
497 +   const fmin = f[0], fmax = f[f.length - 1];
498 +   const X = (v) => (v - fmin) / (fmax - fmin) * w;
499 +   const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
500 +   const det = d.detected, half = det.baud / 2e3;
501 +   g.strokeStyle = "rgba(120,180,255,.35)"; g.setLineDash([4, 4]);
502 +   for (const gf of [det.fc / 1e3 - half, det.fc / 1e3 + half]) {
503 +     g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
504 +   }
505 +   g.setLineDash([]);
506 +   g.strokeStyle = "rgba(255,255,255,.45)";
507 +   g.beginPath(); g.moveTo(X(det.fc / 1e3), 0); g.lineTo(X(det.fc / 1e3), h); g.stroke();
508 +   g.strokeStyle = "#5aa9ff"; g.lineWidth = 1.4; g.beginPath();
509 +   f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
510 +   g.stroke();
511 +   g.fillStyle = "rgba(255,255,255,.5)"; g.font = "10px sans-serif";
512 +   g.fillText(sig(fmin) + " kHz", 4, h - 4);
513 +   const tmax = sig(fmax) + " kHz";
514 +   g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
515 + }
516 +
517 + /* --- waterfall --- */
518 + function drawWaterfall(d) { paintWaterfall($("#fallCanvas"), d.waterfall, 150); }
519 + function paintWaterfall(canvas, wf, hCss) {
520 +   const [g, w, h] = fit(canvas, hCss);
521 +   g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
522 +   if (!wf || !wf.rows) return;
523 +   const lo = pct(wf.db, 0.05), hi = pct(wf.db, 0.95);
524 +   const img = g.createImageData(wf.bins, wf.rows);
525 +   for (let k = 0; k < wf.db.length; k++) {
526 +     const t = Math.max(0, Math.min(1, (wf.db[k] - lo) / (hi - lo + 1e-9)));
527 +     const o = k * 4;

```

// dark -> blue -> white ramp

```

528 +     img.data[o] = t < 0.5 ? 13 + t * 120 : 73 + (t - 0.5) * 364;
529 +     img.data[o + 1] = t < 0.5 ? 19 + t * 260 : 149 + (t - 0.5) * 212;
530 +     img.data[o + 2] = t < 0.5 ? 32 + t * 446 : 255;
531 +     img.data[o + 3] = 255;
532 + }
533 + createImageBitmap(img).then((bmp) => { g.imageSmoothingEnabled = true; g.drawImage(bmp, 0, 0
534 +     , w, h); });
535 + }
536 + /* --- constellation playback (the headline) --- */
537 + const play = { raf: 0, i: 0, on: false, data: null };
538 + let CG = null;    // cached constellation geometry
539 +
540 + function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
541 + function setPlayBtn(on) { $("playBtn").textContent = on ? "⏸ 일시정지" : "▶ 재생"; }
542 +
543 + function startPlay(d) {
544 +     stopPlay();
545 +     play.data = d;
546 +     play.i = 0;
547 +     const n = d.constellation.i.length;
548 +     $("scrub").max = String(Math.max(1, n));
549 +     $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
550 +     drawConstBase(d);
551 +     if (REDUCED) {                                // static full view, no motion
552 +         drawPoints(d, 0, n);
553 +         $("scrub").value = String(n);
554 +         setPlayBtn(false);
555 +         return;
556 +     }
557 +     play.on = true;
558 +     setPlayBtn(true);
559 +     play.raf = requestAnimationFrame(loop);
560 + }
561 + $("playBtn").onclick = () => {
562 +     if (!play.data) return;
563 +     if (play.on) { stopPlay(); setPlayBtn(false); }
564 +     else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
565 + };
566 + $("scrub").oninput = () => {
567 +     if (!play.data) return;
568 +     stopPlay(); setPlayBtn(false);
569 +     play.i = +$("scrub").value;
570 +     drawConstBase(play.data);
571 +     drawPoints(play.data, 0, play.i);    // redraw history up to the scrub point
572 + };
573 +
574 + function loop() {
575 +     const d = play.data, n = d.constellation.i.length;
576 +     const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
577 +     const to = Math.min(n, play.i + step);
578 +     fade();                                // phosphor persistence
579 +     drawPoints(d, play.i, to);
580 +     play.i = to >= n ? 0 : to;
581 +     if (play.i === 0) drawConstBase(d);    // loop: clear the trail
582 +     $("scrub").value = String(play.i);
583 +     if (play.on) play.raf = requestAnimationFrame(loop);
584 + }
585 +
586 + function drawConstBase(d) {

```

```

587 + const [g, w] = fit$("constCanvas");
588 + const fsk = d.family === "fsk";
589 + const I = d.constellation.i, Q = d.constellation.q;
590 + const ideal = fsk ? [] : idealPoints(d.detected.mod);
591 + let lim = 0;
592 + for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
593 + for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
594 + lim = (lim || 1) * 1.12;
595 + const R = w / 2 * 0.9;
596 + CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };
597 +
598 + g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
599 + g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
600 + for (let f = -1; f <= 1; f += 0.5) {
601 +   const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
602 +   g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
603 +   g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
604 + }
605 + if (fsk) { // frequency levels as dashed bands
606 +   g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
607 +   for (const lv of [...new Set(I.map((v) => Math.round(v)))] .filter((v) => Math.abs(v) <= 8)
608 +     ) {
609 +     const [, yy] = CG.map(0, lv);
610 +     g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
611 +   }
612 +   g.setLineDash([]);
613 + } else {
614 +   g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
615 +   for (const p of ideal) {
616 +     const [x, y] = CG.map(p.i, p.q);
617 +     g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
618 +   }
619 + }
620 + function fade() {
621 +   if (!CG) return;
622 +   CG.g.fillStyle = "rgba(13,19,32,.14)"; // translucent wash = phosphor decay
623 +   CG.g.fillRect(0, 0, CG.w, CG.w);
624 + }
625 + function drawPoints(d, from, to) {
626 +   if (!CG || to <= from) return;
627 +   const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
628 +   const fsk = d.family === "fsk";
629 +   const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
630 +   g.globalCompositeOperation = "lighter";
631 +   g.fillStyle = `rgba(90,170,255,${a})`;
632 +   for (let k = from; k < to; k++) {
633 +     // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
634 +     const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
635 +     g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
636 +   }
637 +   g.globalCompositeOperation = "source-over";
638 + }
639 +
640 + /* --- downloads --- */
641 + function saveBlob(name, text, type) {
642 +   const url = URL.createObjectURL(new Blob([text], { type }));
643 +   const a = document.createElement("a");
644 +   a.href = url; a.download = name; a.click();
645 +   URL.revokeObjectURL(url);

```

```

646 + }
647 + const stem = () => state.file.name.replace(/\.^[^.]*/, "");
648 + $("copyBits").onclick = async (e) => { // decoded bitstream -> clipboard, with brief feedback
649 +   const btn = e.currentTarget, was = btn.textContent;
650 +   try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
651 +   catch { btn.textContent = "복사 실패"; }
652 +   setTimeout(() => { btn.textContent = was; }, 1400);
653 + };
654 + $("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
655 + $("dlSyms").onclick = () => {
656 +   const c = state.resp.constellation;
657 +   saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "text/csv");
658 + };
659 + $("dlReport").onclick = () =>
660 +   saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");
661 +
662 + /* --- view helpers --- */
663 + function show(el) { el.classList.remove("hidden"); }
664 + function hideAll() {
665 +   stopPlay();
666 +   ["metaForm", "loading", "errorCard", "results", "batchCard", "surveyCard"]
667 +     .forEach((id) => $(id).classList.add("hidden"));
668 +   $("backBatch").classList.add("hidden");
669 + }
670 + function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }
671 +
672 + /* --- dropzone wiring --- */
673 + const drop = $("dropCard"), input = $("fileInput");
674 + // reset value first: re-picking the SAME file fires no change event otherwise
675 + const pick = () => { input.value = ""; input.click(); };
676 + drop.onclick = pick;
677 + drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick()
678 +   ; } };
679 + input.onChange = () => { if (input.files.length) acceptFiles(input.files); };
680 + drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
681 + drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
682 + drop.addEventListener("drop", (e) => {
683 +   e.preventDefault(); drop.classList.remove("drag");
684 +   if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
685 + });

```